BranchCARET Model Checking of Self Modifying Code

***Abstract*—Binary code analysis is an important task, especially for malware detection. Model checking is one of the static analysis techniques that can reason about behaviour of a program. Pushdown System (PDS) is a natural abstraction used for model checking binary code. However, when working with binary code, it is crucial to take self-modifying instructions into account. We consider Self-Modifying Pushdown Systems (SM-PDS) as an abstraction for self-modifying binary code. To specify malicious behaviour, the language needs to express return values and parameters of functions. Such expressivity is provided by the Temporal Logic of Matching Calls and Returns (CARET). Moreover, the language also needs to express branching behaviour of malware (as in CTL), which is not supported by CARET. BranchCARET is a logic that combines branching features of CTL and matching calls and returns. In this work, we present an efficient algorithm for BranchCARET model checking of *self-modifying* PDS. We tackle the problem by constructing a kind of Büchi PDS and performing a kind of reachability analysis. We have implemented our algorithm in a tool and obtained promising results. In particular, we were able to detect several self-modifying malwares.**

***Index Terms*—Branching CARET model checking, pushdown systems, self modifying code, malware detection.**

I. Introduction

Analysis of binary programs is an important issue when one does not have access to a program's original source code. This is often the case in malware analysis and the analysis of proprietary software. However, binary programs are often obfuscated using self-modification. Therefore, it is crucial to analyze *binary* programs with *self-modification*.

In this work, we consider the verification of self-modifying binary code that modifies itself using self-modifying instructions. Such self-modifying instructions are usually `mov` instructions because they can rewrite the content of the executable code. First, let us demonstrate an example of self-modifying code in Listing 1. This code is supposed to run on an x86 architecture under the Windows operating system.

```
# Address   Bytecode          Assembly
  0x14      eb00              jmp   0x16
  0x16      c6051500000009    mov   [0x15], 0x09
  0x1d      ebf5              jmp   0x14
  0x1f      ff1500000000      call  [InternetOpenA]
```

Listing 1. Example of self-modifying code.

```
# Address   Assembly
  0x14      jmp   0x16      #-- executing  -> jmp 0x1f
                            # mov   [0x15], 0x09
  0x16      mov   [0x15], 0x09
  0x1d      jmp   0x14
  0x1f      call  [InternetOpenA]
```

Listing 2. Example from Listing 1 after self-modification.

The binary program starts execution at the address `0x14`, which contains the bytecode `eb00`. This bytecode corresponds to the assembly instruction `jmp 0x16`, which jumps to the address `0x16`. The address `0x16` contains the binary code `c6051500000009`, which corresponds to the assembly instruction `mov [0x15], 0x09`. This instruction replaces the byte stored at address `0x15` with the byte `0x09`. Note that the address `0x15` corresponds to the byte `00` inside of the bytecode `eb00` stored at address `0x14`. In other words, the instruction `mov [0x15], 0x09` replaces the bytecode `eb00` with the bytecode `eb09`. The new code `eb09` corresponds to the assembly instruction `jmp 0x1f`, which jumps to the address `0x1f`. Thus, the `mov` instruction is a self-modifying instruction that replaces the instruction `jmp 0x16` with the instruction `jmp 0x1f`. After the `mov` instruction, the program executes the address `0x1d`, which contains the bytecode `ebf5`, corresponding to the assembly instruction `jmp 0x14`, which jumps to the address `0x14`. Now, the address `0x14` contains the freshly modified instruction `jmp 0x1f` instead of the original `jmp 0x16`. The binary code after self-modification is presented in Listing 2. The new instruction jumps to the address `0x1f`, which contains the bytecode `ff1500000000`. This bytecode corresponds to the instruction `call [InternetOpenA]`, which calls the Windows API function $InternetOpenA$. This function accesses the internet and opens the provided URL. Malware can use this function to access malicious resources. However, if one analyses this binary without considering self-modification, the instruction calling the $InternetOpenA$ function would appear as unreachable, and the analysis would fail to detect this program as malicious. The program would appear to consist of only the infinite loop at addresses `0x14`, `0x16`, and `0x1d`. Therefore, it is important to consider self-modification when analyzing binary code, because malware can hide the malicious behaviour using self modifications.

To analyze self-modifying binaries, one needs a robust approach. Commonly used binary pattern matching, which relies on specific binary sequences, is not fit to detect evolving malware that changes its syntax without changing the behaviour. On the other hand, dynamic analysis relies on actual execution of malware, but the dynamic analysis can only 'see' one actual behaviour, and not all possible program paths. To analyze binary code more robustly, we consider model checking, which allows to reason about possible behaviours of the program

without executing it. To perform model checking, one requires an adequate abstraction of binary programs. Pushdown System (PDS) is a natural abstraction for binary programs. PDS is a kind of finite automaton equipped with a stack, and PDS were successfully used to model binary programs in [1]. To deal with self-modifying behaviour, we consider Self-Modifying Pushdown Systems (SM-PDS) proposed in [2], which is a kind of PDS that can modify its set of transitions during execution time. Such feature allows to model self-modifying code more naturally. [2] proposed reachability analysis, LTL, and CTL model checking of SM-PDS. However, LTL and CTL is not enough to specify malicious behaviour. It is important to reason about calls and their corresponding returns to have understanding of return values and parameters of functions. The Temporal Logic of Matching Calls and Returns (CARET) [3] was introduced for this reason. Later, [4] proposed CARET model checking of PDS. However, CARET can reason only about 'linear' executions of the program. And there are malicious behaviours that cannot be described with CARET [5]. Consider, for example, a typical behaviour of a spyware, which hunts for sensitive data on the computer by searching files that match certain conditions. If it finds a matching file, then the spyware extracts the sensitive data and continues to search other files in the same fashion. If the search hits a folder, then the spyware performs the same search recursively in the found directory. To achieve this, spyware first calls to the Windows function $FindFirstFileA$ to search for important files. If the call fails, the spyware obtains an error by calling the $GetLastError$ function. If the call succeeds, it returns a handle d to access the search results. The search handle d can point to either a file, or a directory. If d points to a directory, then the spyware looks for important files within that directory using the $FindFirstFileA$ function. If d points to a file, then the malware will try to look for another file matching the same conditions. This is achieved by calling the $FindNextFileA$ function with d as parameter. This behaviour cannot be expressed using neither LTL, nor CTL, nor CARET. On the other hand, CARET with branching time properties can express such behaviour naturally using the following formula: $\mathbf{EF^g}\Big(call(FindFirstFileA) \wedge \mathbf{EX^a} eax = d \wedge \mathbf{AF^g}\big(call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)\big)\Big)$, where $\mathbf{EF^g}$ and $\mathbf{AF^g}$ are the standard CTL operators EF and AF respectively, and $\mathbf{EX^a}$ refers to the abstract successor, i.e. the return location of a call. The subformula $call(FindFirstFileA) \wedge \mathbf{EX^a} eax = d$ states that the program calls the $FindFirstFileA$ function, and when the function returns, the search handle d is stored in the eax register (since return values in x86 binaries are passed through the register eax). Then, the subformula $\mathbf{AF^g}\big(call(GetLastError) \vee call(FindFirstFileA) \vee (call(FindNextFileA) \wedge d\Gamma^*)\big)$ means that in all possible futures, either the $GetLastError$ function is called, or the $FindFirstFileA$ function is called, or the $FindNextFileA$ function called, while $d\Gamma^*$ is also satisfied. The requirement $d\Gamma^*$ expresses that d is located on the top of the stack. Since function parameters are passed on the stack, $d\Gamma^*$ effectively means that d is the first parameter of the function. Therefore, we can specify spyware behaviour using CARET with branching time properties (BranchCARET). Thus, in this work, we reduce the malware detection problem to the model checking problem of SM-PDS against BranchCARET.

In this work, we propose a *direct* and *efficient* algorithm for BranchCARET model checking of self-modifying binary code for malware detection. First, since SM-PDS is a natural abstraction for self-modifying code, we model the binary code as an SM-PDS $\mathcal{P}$. Then, we express the malicious behaviour using BranchCARET formula ψ. Thus, we reduce the problem of malware detection to the BranchCARET model checking of $\mathcal{P}$ over ψ. That is, if $\mathcal{P}$ satisfies ψ, then the binary is malicious. To perform model checking and determine whether $\mathcal{P}$ satisfies ψ, we follow the lines of [5] and [6]. We construct a kind of alternating Büchi pushdown system $\mathcal{BP}$, such that there is an infinite run in $\mathcal{BP}$ that visits accepting control locations infinitely many times only if $\mathcal{P}$ satisfies ψ. This is not trivial for two reasons. First, BranchCARET can reason about *abstract* successors, which refer to the return locations of calls. I.e., if the current step is a call, then the *abstract* successor 'steps over' the inner call and refers directly to the return location. Although this problem was solved for standard non-self-modifying PDS, the problem is harder for SM-PDS. Not only one needs to know the return location, but also, if a self-modification can happen during the inner call, then the return location must reflect the self-modification. The second reason is the *caller* successor in BranchCARET. This kind of successors refers to the latest call *in the past* (often referred to as the 'call-stack'). To reason about the past state, one needs to know if a self-modification happened since this past state. We solve these two problems by introducing a new type of alternating Büchi pushdown systems that keeps track of the states on the 'call-stack'. We achieve this by using a special kind of stack symbols that we call *checkpoints*. If ψ requires to 'go back in time', these checkpoints can be read from the stack and the old state can be restored. Then, to find accepting runs in $\mathcal{BP}$, we introduce an efficient algorithm that constructs a finite-state automaton $\mathcal{A}$ that accepts configurations from which there exists an accepting run.

We have implemented our approach in a tool for malware detection. Our tool takes as input a binary program and abstracts it into an SM-PDS. Then, it takes a BranchCARET formula from a predefined set of formulas that express different malicious behaviours. It performs the BranchCARET model checking and if the SM-PDS model satisfies the BranchCARET formula for malicious behaviour, then it marks the given binary as malicious. We obtained encouraging results, as the tool was able to detect malicious behaviour in several self-modifying malwares, including Spyware, Trojans, and Backdoors. Our tool successfully detected malicious behaviours even in large binaries within a few minutes.

Related Works Model checking of binary code is an extensively studied field with many approaches. One of the

most popular approaches is symbolic execution [7]. Symbolic execution of binaries has been employed in such platforms as BINSEC [8], S2E [9], and Zorya [10]. However, symbolic execution alone is not enough for malware detection, as one needs a robust way to specify features of malware.

One way of describing malicious behaviour is temporal logic, which was used by numerous studies [1], [4], [11]–[14] in the context of malware detection. However, these works do not consider self-modification.

Some studies [15]–[18] proposed formal models for self-modifying code, but such models do not scale to large real world binaries. Control flow reconstruction of self-modifying binaries was proposed by [19], but this technique ignores execution stack. Other studies [20], [21] provide a technique to disassemble packed binaries. However, these works focus solely on packing behaviour, and not on self-modifying instructions. [2] proposed model checking for Self-Modifying Pushdown System (SM-PDS). Nevertheless, in these works, only reachability, LTL, and CTL model checking is considered.

Previously, [5] and [6] simultaneously proposed two branching versions of CARET (BCARET and BranchCARET respectively). We consider in this work BranchCARET since it is more expressive than BCARET, while following the lines of BCARET in expressing malicious behaviour using BranchCARET. However, the authors considered BranchCARET and BCARET model checking only on standard Pushdown Systems, which is not convenient for self-modifying code. Whereas our approach adopts the SM-PDS model and proposes *direct* and *efficient* model checking of SM-PDS over BranchCARET.

Outline Section II introduces the SM-PDS model and how one can abstract a binary code by constructing an SM-PDS. Section III gives the definition of the BranchCARET logic and shows malicious behaviours expressed using BranchCARET. Section IV presents the main contribution - direct and efficient BranchCARET model checking algorithm of SM-PDS. In Section V, we describe the results of practical experiments, including malware detection using BranchCARET model checking. Proofs can be found in the appendix.

II. Modelling Self-Modifying Binary Code as SM-PDS

A. Self-Modifying Pushdown Systems

We define our model for self-modifying binary code. It is a kind of automaton equipped with a stack, and a current set of rules (instructions) that can change during execution (self-modification):

Definition 1. *A Labelled Self-Modifying Pushdown System (SM-PDS) is a tuple $\mathcal{P} = (P, \Gamma, \Delta)$, such that P is a finite set of control locations, Γ is a stack alphabet, and Δ is a finite set of rules of the forms: (1) $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_{int} \langle p', w\rangle$, (2) $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_{call} \langle p', \gamma_1\gamma_2\rangle$, and (3) $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_{ret} \langle p', \varepsilon\rangle$, where $p, p' \in P$, $\gamma, \gamma_1, \gamma_2 \in \Gamma$, $w \in \Gamma^*$, and $\rho, \rho' \subseteq \Delta$.*

To express the self-modifying behaviour of SM-PDS, we introduce the notion of *phase* $\theta \subseteq \Delta$, which is the current set of rules that can be applied. Intuitively, a rule of the form $r = \langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_t \langle p', w\rangle$ for $t \in \{call, int, ret\}$, can be applied only if the current phase θ contains r and every rule from ρ. Such rule pops γ from the stack, pushes the word w onto the stack (w can be the empty word ε), changes the current control location from p to p', removes every rule in ρ from θ, and adds every rule from ρ' to θ. Hence, a rule of the form $\langle p, \gamma\rangle \xhookrightarrow{(\emptyset,\emptyset)}_t \langle p', w\rangle$ is a standard PDS rule that does not perform self-modification. We omit $(\emptyset, \emptyset)$ in such cases, denoting the rule as $\langle p, \gamma\rangle \hookrightarrow_t \langle p', w\rangle$. A rule of the form $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_{call} \langle p', \gamma_1\gamma_2\rangle$ often corresponds to a call statement of the form $\gamma \xrightarrow{call\ proc} \gamma_2$, where $proc$ is the name of the called procedure, γ is the location of the call statement, γ_1 is the entry location of $proc$, γ_2 is the return location, p and p' represent global state of the program, which contains global variables and return values. A rule of the form $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_{ret} \langle p', \varepsilon\rangle$ corresponds to a return statement, and a rule of the form $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_{int} \langle p', w\rangle$ corresponds to an internal statement within a procedure.

For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$, a *configuration* $c \in P \times \Gamma^* \times 2^\Delta$ of $\mathcal{P}$ is a tuple of the form $(\langle p, \omega\rangle, \theta)$, where $p \in P$ is the current control location, $\omega \in \Gamma^*$ is the current stack content, and $\theta \subseteq \Delta$ is the current phase. Let $Conf$ be the set of all possible configurations of $\mathcal{P}$. For configurations c, c', and a label $t \in \{call, int, ret\}$, the successor relation $c \overset{t}{\Longrightarrow} c'$ is defined as follows. For any $t \in \{call, int, ret\}$, $p, p' \in P$, $\omega \in \Gamma^*$, $\theta, \rho, \rho' \subseteq \Delta$, and $\theta' = (\theta \backslash \rho) \cup \rho'$:

- if $\langle p, \gamma\rangle \xhookrightarrow{(\rho,\rho')}_t \langle p', v\rangle \in \theta$ and $\rho \subseteq \theta$, then $(\langle p, \gamma\omega\rangle, \theta) \overset{t}{\Longrightarrow} (\langle p', v\omega\rangle, \theta')$.

A *run* π on $\mathcal{P}$ is an infinite sequence of the form $\pi = (c_0, t_0)(c_1, t_1)\ldots$, where for every $i \geq 0$, $c_i \in Conf$, and $\exists t_i \in \{call, int, ret\}$, s.t. $c_i \overset{t_i}{\Longrightarrow} c_{i+1}$. We denote by $\pi(i)$ the i-th configuration c_i of π. We say that π starts with c_0. Let $Ext(\pi, i)$ be the set of runs on $\mathcal{P}$, s.t. for each $\pi' \in Ext(\pi, i)$, π' is an extension of π from i, i.e. for each $j \leq i$, $\pi'(j) = \pi(j)$.

Let $\pi = (c_0, t_0)(c_1, t_1)\ldots$ be a run on $\mathcal{P}$, such that for i, $i \geq 0$, $\pi(i) = c_i = (\langle p_i, \omega_i\rangle, \theta_i)$, where $p_i \in P$, $\omega_i \in \Gamma^*$, $\theta_i \subseteq \Delta$. Each index $i \geq 0$ on π may have three types of successors:

- the *global successor*, $next^g_\pi(i) = i + 1$;
- the *abstract successor*, $next^a_\pi(i)$, depends on the transition $c_i \overset{t_i}{\Longrightarrow} c_{i+1}$:
  - if $t_i = call$, then $next^a_\pi(i)$ is the index of the matching ret,
  - if $t_i = int$, then $next^a_\pi(i) = i + 1$, and
  - if $t_i = ret$, then $next^a_\pi(i) = \perp$;
- the *caller successor*, $next^c_\pi(i)$, is the index of the latest unmatched $call$ in the past ($next^c_\pi(i) < i$), otherwise

$next^c_\pi(i) = \bot$.

Intuitively, a *global* successor is a "standard successor". An *abstract* successor depends on the nature of the transition $c_i \xrightarrow{t_i} c_{i+1}$. If it is a *call*, then the abstract successor is the corresponding return location. If it is an internal transition, then the abstract successor is the global successor. If it is a return, then the abstract successor does not exist ($\bot$). The *caller* successor corresponds to the call site of the current procedure. Note that sometimes $\pi(i)$ might not have an abstract (resp. caller) successor. In these cases, we say that $next^a_\pi(i)$ (resp. $next^c_\pi(i)$) is undefined, or $next^a_\pi(i) = \bot$ (resp. $next^c_\pi(i) = \bot$).

B. Modelling Self-Modifying Binary Code as SM-PDS

This section presents how one can model a self-modifying code as SM-PDS. Our approach follows the lines of [2]. We focus on x86 architecture, but a similar approach can be derived for other architectures. We also focus on the self-modification achieved solely by mov instructions. We suppose that we are given an oracle O, which constructs a Control Flow Graph (CFG) from the given binary. Such an oracle may be obtained using such tools as IDA Pro [22], Radare2 [23], or ANGR [24]. A binary address in the program corresponds to a node in the CFG, and a binary instruction is an edge in the CFG. For example, an edge of the form $d_1 \xrightarrow{inst} d_2$ represents an instruction $inst$ located at the binary address d_1, and d_2 is the binary address of the next instruction. Let Γ be the set of values that can be pushed onto the stack (it can be obtained from the oracle O). Then, for each edge in the CFG and $\gamma \in \Gamma$, we construct the rules of the SM-PDS as follows:

- For $d_1 \xrightarrow{call\ d_2} d_3$, we create a rule $\langle d_1, \gamma\rangle \hookrightarrow_{call} \langle d_2, d_3\gamma\rangle$, i.e. we push the return location d_3 to the stack and go to the entry location d_2 of the called function. In contrast to 'standard' PDS construction for programs, where the current program point is the topmost symbol on the stack, in our construction, the current instruction pointer of the program (stored in the eip register) corresponds to the control location of the SM-PDS, and the stack simulates the actual stack of the program.
- For $d_1 \xrightarrow{ret} d_3$, we create a rule $\langle d_1, \gamma\rangle \hookrightarrow_{ret} \langle \gamma, \varepsilon\rangle$, i.e. we pop the return location γ from the stack, and set the current control location to γ.
- For $d_1 \xrightarrow{push\ \alpha} d_2$, we create a rule $\langle d_1, \gamma\rangle \hookrightarrow_{int} \langle d_2, \alpha\gamma\rangle$, i.e. we push α onto the stack.
- For $d_1 \xrightarrow{pop\ r} d_2$, we create a rule $\langle d_1, \gamma\rangle \hookrightarrow_{int} \langle d_2, \varepsilon\rangle$, i.e. we pop γ from the stack.
- For $d_1 \xrightarrow{mov\ d,v} d_2$, where d is an address of the program that belongs to the executable part of the binary, then it is a self-modifying instruction. Let ρ_1 be the set of rules obtained from the address d using this construction. Let ρ_2 be the set of rules obtained from the address d after executing $mov\ d, v$. Then, we create the SM-PDS rule $\langle d_1, \gamma\rangle \xhookrightarrow{(\rho_1,\rho_2)}_{int} \langle d_2, \gamma\rangle$.
- For the other edges of the form $d_1 \xrightarrow{inst} d_2$, we create a rule of the form $\langle d_1, \gamma\rangle \hookrightarrow_{int} \langle d_2, \gamma\rangle$.

The initial configuration is $(\langle d_0, \#\rangle, \theta_0)$, where d_0 is the entry point of the binary, $\#$ is the 'bottom-of-the-stack' symbol that cannot be popped, and θ_0 is the set of rules obtained from this procedure before executing any self-modifying instructions.

III. Specifying Malicious Behaviour with BranchCARET

Given a set of atomic propositions AP, we define a BranchCARET formula ψ over AP as follows:

Definition 2. *A BranchCARET formula ψ is ($a \in AP$ and $b \in \{g, a, c\}$):*

$$\psi ::= a \mid \neg a \mid \psi \vee \psi \mid \psi \wedge \psi \mid EX^b\psi \mid AX^b\psi \mid E[\psi U^b \psi] \mid A[\psi U^b \psi] \mid E[\psi R^b \psi] \mid A[\psi R^b \psi]$$

For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$ and a BranchCARET formula ψ, let λ be a labelling function $\lambda : P \to 2^{AP}$ that labels each control location with a set of atomic propositions. Consider a run $\pi = (c_0, t_0)(c_1, t_1)\dots$ on $\mathcal{P}$, s.t. for $i \geq 0$, $c_i = (\langle p_i, w_i\rangle, \theta_i)$. We write $(\pi, i) \models \psi$ if π satisfies ψ at the i-th step:

1) $(\pi, i) \models e$, where $e \in AP$, if $e \in \lambda(p_i)$;
2) $(\pi, i) \models \neg e$, where $e \in AP$, if $e \notin \lambda(p_i)$;
3) $(\pi, i) \models \psi_1 \vee \psi_2$ if $(\pi, i) \models \psi_1$ or $(\pi, i) \models \psi_2$;
4) $(\pi, i) \models \psi_1 \wedge \psi_2$ if $(\pi, i) \models \psi_1$ and $(\pi, i) \models \psi_2$;
5) $(\pi, i) \models EX^b\psi$, where $b \in \{g, a, c\}$, if there exists $\pi' \in Ext(\pi, i)$, s.t. $next^b_{\pi'}(i) \neq \bot$ and $(\pi', next^b_{\pi'}(i)) \models \psi$;
6) $(\pi, i) \models AX^b\psi$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$, $next^b_{\pi'}(i) \neq \bot$ implies $(\pi', next^b_{\pi'}(i)) \models \psi$;
7) $(\pi, i) \models E[\psi_1 U^b \psi_2]$, where $b \in \{g, a, c\}$, if there exists $\pi' \in Ext(\pi, i)$, s.t. there is a sequence $s = s_0 s_1 \dots s_k$, where $s_0 = i$, for j, $0 \leq j < k$, $s_{j+1} = next^b_{\pi'}(s_j)$, $(\pi', s_j) \models \psi_1$, and $(\pi', s_k) \models \psi_2$;
8) $(\pi, i) \models A[\psi_1 U^b \psi_2]$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$, there is a sequence $s = s_0 s_1 \dots s_k$, where $s_0 = i$, for j, $0 \leq j < k$, $s_{j+1} = next^b_{\pi'}(s_j)$, $(\pi', s_j) \models \psi_1$, and $(\pi', s_k) \models \psi_2$;
9) $(\pi, i) \models E[\psi_1 R^b \psi_2]$, where $b \in \{g, a, c\}$, if there exists $\pi' \in Ext(\pi, i)$ and a sequence $s = s_0 s_1 \dots$, where $s_0 = i$ and for j, $j \geq 0$, $s_{j+1} = next^b_{\pi'}(s_j)$, we have that $(\pi', s_j) \not\models \psi_2$ implies that $\exists k < j$, s.t. $(\pi', s_k) \models \psi_1$;
10) $(\pi, i) \models A[\psi_1 R^b \psi_2]$, where $b \in \{g, a, c\}$, if for all $\pi' \in Ext(\pi, i)$, there is a sequence $s = s_0 s_1 \dots$, s.t. $s_0 = i$ and for j, $j \geq 0$, $s_{j+1} = next^b_{\pi'}(s_j)$, we have that $(\pi', s_j) \not\models \psi_2$ implies that $\exists k < j$, s.t. $(\pi', s_k) \models \psi_1$.

We say that $\pi \models \psi$ if $(\pi, 0) \models \psi$. For a configuration c of $\mathcal{P}$, $c \models \psi$ iff there exists a run π on $\mathcal{P}$ starting from c, s.t. $\pi \models \psi$. Let $Cl(\psi)$ be the closure of ψ, i.e. the set of all subformulas of ψ.

Intuitively, EX^g and AX^g correspond to *global* successors, EX^a and AX^a correspond to *abstract* successors, EX^c and AX^c correspond to *caller* successors. $E[\psi U^a \psi]$ and $A[\psi U^a \psi]$ (resp. $E[\psi U^c \psi]$ and $A[\psi U^c \psi]$) are *exists until* and *for all until* operators that consider only *abstract* (resp. *caller*) successors. $E[\psi R^a \psi]$ and $A[\psi R^a \psi]$ (resp. $E[\psi R^c \psi]$ and $A[\psi R^c \psi]$) are *exists release* and *for all release* operators that consider only

abstract (resp. *caller*) successors. A BranchCARET formula with only operators labelled by g is equivalent to the same CTL formula (with g omitted).

A. Specifying Malicious Behaviour with BranchCARET

In this section, we show how BranchCARET is needed to specify several malicious behaviours. The malicious behaviours presented in this section cannot be described by CTL or CARET alone. Therefore, it is important to consider BranchCARET model checking of self-modifying code. We use operators of the forms $\mathbf{EG^g}\phi$ (resp. $\mathbf{EG^a}\phi$ and $\mathbf{EG^c}\phi$) meaning that ϕ holds *globally* on a run, i.e. $\mathbf{E}[False\mathbf{R^g}\phi]$ (resp. $\mathbf{E}[False\mathbf{R^a}\phi]$ and $\mathbf{E}[False\mathbf{R^c}\phi]$). We use operators of the forms $\mathbf{EF^g}\phi$ (resp. $\mathbf{EF^a}\phi$ and $\mathbf{EF^c}\phi$), meaning that ϕ holds *eventually* on a run, i.e. $\mathbf{E}[True\mathbf{U^g}\phi]$ (resp. $\mathbf{E}[True\mathbf{U^a}\phi]$ and $\mathbf{E}[True\mathbf{U^c}\phi]$).

1) Obtaining malware's filename: Common behaviour for different malware is the ability to obtain the filename of the program during the execution time, so that it can be used to copy itself in other locations. This is achieved by calling the $GetModuleFileNameA$ function with 0 as parameter. Such behaviour was described by [25] using the following formula: $call(GetModuleFileNameA) \wedge 0\Gamma^*$. The $0\Gamma^*$ proposition specifies that 0 is on top of the stack (since parameters are usually passed on the stack in assembly). This formula perfectly represents obtaining the filename if we consider 'standard' programming policies. However, malware can obfuscate calls to system functions using such techniques as in this code:

```
d0:   push 0
d1:   call [d]
d2:   ...
d :   jmp [GetModuleFileNameA]
```

In fact, every malware we considered uses such call obfuscation. For such cases, there is no direct call to $GetModuleFileNameA$, and at address `d`, where the `jmp [GetModuleFileNameA]` instruction is located, the topmost symbol on the stack is the return location `d2`, and not 0. Indeed, it is guaranteed that 0 is on top of the stack only at the call location `d1`. Thus, to represent this behaviour in a more accurate way, we need the following BranchCARET formula: $GetModuleFileNameA \wedge \mathbf{EX^c}0\Gamma^*$, where $GetModuleFileNameA$ refers to the entry location of the $GetModuleFileNameA$ function, and the $\mathbf{EX^c}$ refers to the caller successor, i.e. the point where the call is made. The caller successor corresponds to the instruction `call [GetModuleFileNameA]` if the 'standard' call was made, or, if the code is obfuscated, to the `call [d]` instruction. We apply the same reasoning for every system call in malicious behaviours described in this section. That is, to describe $call(F)$ with h as parameter, we use the following BranchCARET formula: $F \wedge \mathbf{EX^c}h\Gamma^*$ to specify that that when a function F is called, h is on the top of the stack.

2) Registry key injection: Malware employs registry key injection to get executed on the infected machine even after the system restarts. This is usually achieved by obtaining the filename of the malware using the $GetModuleFileNameA$ function, and injecting it into the system registry using the $RegSetValueExA$ function. Registry key injection can be described using the following BranchCARET formula:

$$\psi_r = \mathbf{EF^g}(GetModuleFileNameA \wedge \mathbf{EX^c}0\Gamma^* \wedge \mathbf{EX^c}\mathbf{EX^a}\mathbf{EF^g}RegSetValueExA)$$

The subformula $GetModuleFileNameA \wedge \mathbf{EX^c}0\Gamma^*$ was explained previously. It means that the $GetModuleFileNameA$ function is called with 0 as parameter. To specify the registry key injection truthfully, we need to specify that the $RegSetValueExA$ function is called *after* $GetModuleFileNameA$ returns. The return location of a function is the next instruction after the call is made, i.e. the abstract successor ($\mathbf{EX^a}$) of the caller successor ($\mathbf{EX^c}$). Therefore, we can use the construct $\mathbf{EX^c}\mathbf{EX^a}$ to refer to the return location of $GetModuleFileNameA$. In the following formulas, we will use $F \wedge \mathbf{EX^c}\mathbf{EX^a}\phi$ to specify that ϕ must be satisfied after the function F returns. The formula $\mathbf{EF^g}RegSetValueExA$ specifies that in the future, the $RegSetValueExA$ function is called. Thus, ψ_r expresses that the $GetModuleFileNameA$ function is called with 0 as parameter, and after the function returns, the program calls the $RegSetValueExA$ function.

3) Trojan downloader behaviour: Trojan downloaders are programs that are designed to deliver other malicious files to the target computer. They download the malicious files from the network and execute them immediately. To download files, programs must open a socket using the $socket$ function, and then receive the files using the $recv$ function, which takes the opened socket as parameter. Then, the downloaded file gets executed using the $CreateProcess$ function. This behaviour can be described using the following formula (we consider the disjunction $\bigvee_{d\in\Gamma}$ over all values d possibly located on the stack):

$$\psi_t = \bigvee_{d\in\Gamma} \mathbf{EF^g}\Big(socket \wedge \mathbf{EX^c}\mathbf{EX^a}\big(eax = d \wedge \mathbf{EF^g}(recv \wedge \mathbf{EX^c}d\Gamma^* \wedge \mathbf{EX^c}\mathbf{EX^a}\mathbf{EF^g}CreateProcess)\big)\Big)$$

Intuitively, this formula describes that in the future, $socket \wedge \mathbf{EX^c}\mathbf{EX^a}\big(eax = d \wedge \mathbf{EF^g}(recv \wedge \mathbf{EX^c}d\Gamma^* \wedge \mathbf{EF^g}CreateProcess)\big)$ is satisfied. Subformula $socket \wedge \mathbf{EX^c}\mathbf{EX^a}eax = d$ means that the $socket$ function is called, and at the return location of $socket$ (i.e. at the abstract successor of the call), the value of the register eax is equal to d. Usually, the value of the eax register right after returning from a function contains the return value of the function. That is, the subformula means that the return value of the $socket$ function (the opened socket) is equal to d. Then, the subformula $recv \wedge \mathbf{EX^c}d\Gamma^*$ means that in the future, the $recv$ function is called with d as parameter (in the same way as 0 for $GetModuleFileNameA$ in the registry key injection formula ψ_r). Then, the subformula $\mathbf{EF^g}CreateProcess$ states that the $CreateProcess$ function will be called in the future.

4) Spyware behaviour: [5] introduced the BCARET formula describing spyware behaviour. The spyware behaviour and the BCARET formula specifying the behaviour were explained in the introduction. However, the given formula does not take into account possible call obfuscations. To tackle it, we present a more accurate BranchCARET formula for spyware behaviour:

$$\psi_s = \bigvee_{d\in\Gamma} \mathbf{EF^g}\Big(FindFirstFileA \wedge \mathbf{EX^c EX^a}\Big(eax = d \wedge \mathbf{AF^g}\big(GetLastError \vee FindFirstFileA \vee (FindNextFileA \wedge \mathbf{EX^c}d\Gamma^*)\big)\Big)\Big)$$

This formula states that in the future, the formula $FindFirstFileA \wedge \mathbf{EX^c EX^a}\big(eax = d \wedge \mathbf{AF^g}\big(GetLastError \vee FindFirstFileA \vee (FindNextFileA \wedge \mathbf{EX^c}d\Gamma^*)\big)\big)$ is satisfied. The subformula $FindFirstFileA \wedge \mathbf{EX^c EX^a}eax = d$ states that the $FindFirstFileA$ function is called, and at the return location from $FindFirstFileA$, the value of the eax register is equal to d. Then, subformula $\mathbf{AF^g}\big(GetLastError \vee FindFirstFileA \vee (FindNextFileA \wedge \mathbf{EX^c}d\Gamma^*)\big)$ states that in all possible futures, either the $GetLastError$ function is called, the $FindFirstFileA$ function is called, or the formula $FindNextFileA \wedge \mathbf{EX^c}d\Gamma^*$ is satisfied, which states that the $FindNextFileA$ function is called with d as its parameter.

5) Anti-antivirus behaviour: Some malware attempts to evade antivirus programs by first shutting down the antivirus and then executing the malicious payload. To terminate the antivirus process, first, malware obtains the list of currently running processes using the $CreateToolhelp32Snapshot$ function, which returns a handle d to the list. Then, it calls the $Process32First$ function with d as parameter to get the first process from the list. Then, it iterates over the list by calling the $Process32Next$ function with d as parameter in a loop. Then, if it finds the antivirus program, it calls the $OpenProcess$ function to obtain access to the antivirus program. Then, the malware shuts down the antivirus by calling the $TerminateProcess$ function. Such behaviour can be described using the following formula:

$$\begin{aligned}\psi_k = \bigvee_{d\in\Gamma} \mathbf{EF^g}\Big(& CreateToolhelp32Snapshot \\ &\wedge \mathbf{EX^c EX^a}\Big(eax = d \wedge \mathbf{EF^g}\big(Process32First \wedge \mathbf{EX^c}d\Gamma^* \\ &\wedge \mathbf{AG^g EF^g}(Process32Next \wedge \mathbf{EX^c}d\Gamma^* \\ &\wedge \mathbf{EF^g}[OpenProcess \wedge \mathbf{AF^g}TerminateProcess])\big)\Big)\Big)\end{aligned}$$

The formula states that eventually, the subformula $CreateToolhelp32Snapshot \wedge \mathbf{EX^c EX^a}eax = d$ will be satisfied. This subformula states that the $CreateToolhelp32Snapshot$ function is called, and the return value of the function is equal to d. Then, the subformula $Process32First \wedge \mathbf{EX^c}d\Gamma^*$ states that the $Process32First$ function is called with d as parameter. Then, the subformula $\mathbf{AG^g EF^g}\big(Process32Next \wedge \mathbf{EX^c}d\Gamma^* \wedge \mathbf{EF^g}(OpenProcess \wedge \mathbf{AF^g}TerminateProcess)\big)$ states that there is a loop ($\mathbf{AG^g EF^g}$), in which the function $Process32Next$ is called with d as parameter, and in the future, the subformula $\mathbf{EF^g}(OpenProcess \wedge \mathbf{AF^g}TerminateProcess)$ is satisfied. This subformula describes that in the future, the $OpenProcess$ function is called, and after that, in all possible runs the $TerminateProcess$ function is called.

IV. BranchCARET Model Checking of SM-PDSs

Given a binary code and a malicious behaviour described by a BranchCARET formula ψ, to determine whether the binary code involves this behaviour ψ, we first model the program as an SM-PDS $\mathcal{P}$ as described in Section II-B. Then, we reduce the problem of determining whether $\mathcal{P}$ contains this malicious behaviour or not to the model checking problem of $\mathcal{P}$ against ψ. To this aim, we compute a kind of Alternating Self-Modifying Büchi PDS, which is a kind of product between the SM-PDS $\mathcal{P}$ and the BranchCARET formula ψ, such that $\mathcal{P} \models \psi$ iff there is an accepting run in $\mathcal{BP}$ that visits accepting control locations infinitely many times. We fix an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$ and a BranchCARET formula ψ defined over a set of atomic propositions AP with a labelling function λ.

From now on, for a rule of the form $\langle p, \gamma\rangle \xrightarrow{(\rho,\rho')}_{call} \langle p', \gamma_1\gamma_2\rangle$, we will refer to γ_2 as to the 'return location', which is different from the binary code's semantics (where the return location would be γ_1).

A. Self-Modifying Alternating Büchi Pushdown System with Checkpoints

BranchCARET model checking introduces a significant complication in the model checking process, since a formula can talk about *caller successors*. Since caller successors are in the past of the current configuration, one needs a way to obtain previous states from the current configuration. To store information about previous states, we introduce the notion of *checkpoints*, which are a special kind of stack symbols denoted as χ. We define Self-Modifying Alternating Büchi Pushdown Systems with Checkpoints as follows:

Definition 3. *A Self-Modifying Alternating Büchi Pushdown System with Checkpoints (SM-ABPDS) is a tuple* $\mathcal{BP} = (P', \Gamma', X, \Delta', restore, discard, \delta, F)$*, where* P' *is a finite set of control locations,* Γ' *is a stack alphabet,* X *is a checkpoint alphabet,* Δ' *is a set of alternating rules,* $restore$ *is a set of rules that restore phase from a checkpoint,* $discard$ *is a set of rules that discard phases from checkpoints,* δ *is a phase alphabet, and* F *is an accepting set of control locations.*

Rules in Δ' *have the following form:* $p\gamma \xrightarrow{[\sigma_1,\ldots,\sigma_h]} \{p_1v_1, \ldots, p_hv_h\}$*, where* $p \in P'$*,* $\gamma \in \Gamma'$*, for* $1 \le i \le h$*,* σ_i *has the form* $(\rho_i, \rho_i') \in 2^\delta \times 2^\delta$*,* $p_i \in P'$*, and either* $v_i \in \Gamma'^*$ *or* $v_i \in \Gamma' \times X$*.*

Rules in $restore$ and $discard$ have the form $p\chi \hookrightarrow p'\gamma$, where $p, p' \in P'$, $\chi \in X$, and $\gamma \in \Gamma'$.

A phase θ of $\mathcal{BP}$ is a set $\theta \subseteq \delta$ that consists of symbols from the phase alphabet δ. A checkpoint χ, when on the stack, is always paired with a phase θ in a symbol of the form χ^θ. The set of possible symbols on the stack of $\mathcal{BP}$ is $G = \Gamma' \cup (X \times 2^\delta)$. Let us define a function $em : 2^\delta \times (\Gamma'^* \cup (\Gamma' \times X)) \to G^*$, defined as $em(\theta, v) = v$ for $v \in \Gamma'^*$, and $em(\theta, \gamma\chi) = \gamma\chi^\theta$ for $\gamma\chi \in \Gamma' \times X$. The em function embeds the phase θ into the checkpoints of the word v. As will be shown later, this allows to come back to the phase at which the call (of the caller successor) was made. Formally, a configuration of $\mathcal{BP}$ is a tuple $(\langle p, w\rangle, \theta) \in P \times G^* \times 2^\delta$. Let the set of all configurations of $\mathcal{BP}$ be $Conf_{\mathcal{BP}}$. For a configuration c and set of configurations $S \subseteq Conf_{\mathcal{BP}}$, we define the successor relation $c \Longrightarrow S$ as follows:

1) if $p\gamma \xhookrightarrow{[\sigma_1,\ldots,\sigma_h]} \{p_1v_1, \ldots, p_hv_h\} \in \Delta'$, where for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$, then $(\langle p, \gamma\omega\rangle, \theta) \Longrightarrow \{\big(\langle p_i, em(\theta, v_i)\omega\rangle, (\theta \backslash \rho_i) \cup \rho'_i\big) \mid \text{if } \rho_i \subseteq \theta\}$;
2) if $p\chi \hookrightarrow p'\gamma \in restore$, then $(\langle p, \chi^\theta\omega\rangle, \theta') \Longrightarrow \{(\langle p', \gamma\omega\rangle, \theta)\}$ for any $\theta' \subseteq \delta$;
3) if $p\chi \hookrightarrow p'\gamma \in discard$, then $(\langle p, \chi^\theta\omega\rangle, \theta') \Longrightarrow \{(\langle p', \gamma\omega\rangle, \theta')\}$ for any $\theta' \subseteq \delta$;

Consider an SM-ABPDS with checkpoints $\mathcal{BP} = (P', \Gamma', X, \Delta', restore, discard, \delta, F)$. The idea behind $\mathcal{BP}$ is that the execution of $\mathcal{BP}$ should correspond to the execution of $\mathcal{P}$. To do so, $\mathcal{BP}$ needs information about the phase in $\mathcal{P}$. However, since the phase in $\mathcal{P}$ consists of rules from Δ, and $\mathcal{BP}$ has a different set of rules Δ', then $\mathcal{BP}$ needs to synchronise with the current phase of $\mathcal{P}$. To this aim, we introduce the *phase alphabet* δ, which represents symbols that can be in the phase of $\mathcal{BP}$. Later, δ will correspond to Δ in $\mathcal{P}$ and the phase $\theta \subseteq \delta$ of $\mathcal{BP}$ will then correspond directly to the phase $\theta \subseteq \Delta$ of $\mathcal{P}$.

Intuitively, given a configuration $(\langle p, \gamma\omega\rangle, \theta)$ of $\mathcal{BP}$, application of a rule of the form $p\gamma \xhookrightarrow{[\sigma_1,\ldots,\sigma_h]} \{p_1v_1, \ldots, p_hv_h\} \in \Delta'$ corresponds to applying several SM-PDS rules of the forms $p\gamma \xhookrightarrow{(\rho_i,\rho'_i)} p_iv_i$ for $1 \le i \le h$, where $\sigma_i = (\rho_i, \rho'_i)$, at the same time. $\mathcal{BP}$ transitions into several configurations at once (it is alternating) and the current configuration has a *set* of successors (at most h). We say that such rule allows h possible transitions. For $1 \le i \le h$, the label $\sigma_i = (\rho_i, \rho'_i)$ specifies (1) which rules must be in the current phase for the i-th transition, and (2) the self-modification performed by the i-th transition. The i-th transition takes place only if $\rho_i \subseteq \theta$, and it changes the control location from p to p_i, pops γ from the stack, removes every rule in ρ_i from θ, and adds every rule from ρ'_i to θ. Then it pushes a word $v'_i = em(\theta, v_i)$ onto the stack. v'_i depends on v_i, which can be of two forms:

- if $v_i \in \Gamma'^*$ (it does not contain checkpoints), then $v_i = v'_i$,
- if $v_i = \gamma'\chi \in \Gamma' \times X$, where χ is a checkpoint, then $v'_i = \gamma'\chi^\theta$ (this allows to memorize the phase on which the checkpoint was created).

Therefore, the i-th successor is a configuration $(\langle p_i, v'_i\omega\rangle, \theta_i)$, where $\theta_i = (\theta \backslash \rho_i) \cup \rho'_i$. If, for some i, $\sigma_i = (\emptyset, \emptyset)$, then, this means that the current configuration can perform the i-th transition independent of its phase. In that case, for succinctness, we say that $\sigma_i = \top$.

Once a checkpoint χ^θ is pushed onto the stack, it cannot be popped using rules from Δ' because they require a symbol $\gamma \in \Gamma'$ on top of the stack. We introduce $restore$ and $discard$ rules that deal with checkpoints. A $restore$ rule of the form $p\chi \xhookrightarrow{restore} p'\gamma$ pops χ^θ from the stack for any θ, sets the current phase to θ, changes the control location from p to p', and pushes γ onto the stack. A $discard$ rule of the form $p\chi \xhookrightarrow{discard} p'\gamma$ pops χ^θ from the stack for any θ, changes the control location from p to p', and pushes γ onto the stack (without changing the current phase).

A run η on $\mathcal{BP}$ starting from configuration $c \in Conf_{\mathcal{BP}}$ is a tree with each node labelled by a configuration of $\mathcal{BP}$. The root is labelled with c, and a node labelled with c' has the set of children S if $c' \Longrightarrow S$ for some $S \subseteq Conf_{\mathcal{BP}}$. A run η on $\mathcal{BP}$ is *accepting* if every branch of η visits accepting control locations from F infinitely many times.

B. From BranchCARET Model Checking Problem to the Emptiness Problem of SM-ABPDS

In this section, we will show how we can reduce the problem of BranchCARET model checking of SM-PDS to the problem of finding accepting runs in an SM-ABPDS. Let $Cl^a(\psi) = \{QX^a\phi_1, Q[\phi_1 U^a \phi_2], Q[\phi_1 R^a \phi_2] \in Cl(\psi) \mid Q \in \{E, A\}\}$ and $Cl^c(\psi) = \{QX^c\phi_1, Q[\phi_1 U^c \phi_2], Q[\phi_1 R^c \phi_2] \in Cl(\psi) \mid Q \in \{E, A\}\}$ We can construct an SM-ABPDS $\mathcal{BP} = (P', \Gamma', X, \Delta', restore, discard, \Delta, F)$, where:

- $P' = P'_g \cup P'_a \cup P'_c$, where:
  - $P'_g = P \times Cl(\psi)$ is a set of control locations of the form $[p, \phi]$,
  - $P'_a = P \times Cl^a(\psi) \times \{r\}$ is a set of control locations of the form $[p, \phi]^r$,
  - $P'_c = \{p^c\} \times Cl^c(\psi)$ is a set of control locations of the form $[p^c, \phi]$, s.t. p^c is a new state;
- $\Gamma' = \Gamma \cup (\Gamma \times \{r\})$, s.t. $\Gamma \times \{r\}$ is the set of stack symbols of the form γ^r;
- $X = \Gamma \times P \times \Gamma$;
- $F = \{[p, e], [p, \neg e] \mid e \in AP\} \cup F^g \cup F^a \cup F^c$, where:
  - $F^g = \{[p, \phi] \in P'_g \mid \phi = Q[\phi_1 R^g \phi_2]$ for $Q \in \{E, A\}\}$,
  - $F^a = \{[p, \phi]^r \in P'_a \mid \phi = Q[\phi_1 R^a \phi_2]$ for $Q \in \{E, A\}$ or $\phi = AX^a\phi_1\}$,
  - $F^c = \{[p^c, \phi] \in P'_c \mid \phi = Q[\phi_1 R^c \phi_2]$ for $Q \in \{E, A\}$ or $\phi = AX^c\phi_1\}$.

Without loss of generality, we assume that Γ contains a symbol $\#$ that is always located at the bottom of the stack and can never be popped. Moreover, for any rule of the form $r = p\gamma \xhookrightarrow{(\rho,\rho')}_t p'w \in \Delta$, w.l.o.g. we assume that $r \in \rho$. We can assume this because any rule $r = p\gamma \xhookrightarrow{(\rho,\rho')}_t p'w \in \Delta$, where $r \notin \rho$, can be effectively translated to an equivalent

rule of the form $p\gamma \xrightarrow{(\rho\cup\{r\},\rho'\cup\{r\})}_t p'w$. Therefore, for $r = p\gamma \xrightarrow{(\rho\cup\{r\},\rho'\cup\{r\})}_t p'w \in \Delta$ to be applied at the phase θ, it is sufficient to assure that $\rho \cup \{r\} \subseteq \theta$.

We denote by $True$ the atomic proposition that is satisfied by every control location in P, and we denote by $False$ the atomic proposition that is never satisfied. The SM-ABPDS rules Δ', $restore$, and $discard$ are defined as follows. For $p \in P$, $\gamma \in \Gamma$, $\phi \in Cl(\psi)$:

1) if $\phi = a \in AP$ and $a \in \lambda(p)$, then $[p, a]\gamma \xhookrightarrow{[\top]} \{[p, a]\gamma\}$;
2) if $\phi = \neg a \in AP$ and $a \notin \lambda(p)$, then $[p, \neg a]\gamma \xhookrightarrow{[\top]} \{[p, \neg a]\gamma\}$;
3) if $\phi = \phi_1 \vee \phi_2$, then $[p, \phi]\gamma \xhookrightarrow{[\top]} \{[p, \phi_1]\gamma\}$ and $[p, \phi]\gamma \xhookrightarrow{[\top]} \{[p, \phi_2]\gamma\} \in \Delta'$;
4) if $\phi = \phi_1 \wedge \phi_2$, then $[p, \phi_1 \wedge \phi_2]\gamma \xhookrightarrow{[\top,\top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$;
5) if $\phi = EX^g\phi'$, then $[p, EX^g\phi']\gamma \xhookrightarrow{[\sigma]} \{[p', \phi']w\}$, s.t. either $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$, $p\gamma \xrightarrow{(\rho,\rho')}_{ret} p'w \in \Delta$, or $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$ and $w = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$;
6) if $\phi = AX^g\phi'$, then $[p, AX^g\phi']\gamma \xhookrightarrow{[\sigma_1,\ldots,\sigma_h]} \{[p_1, \phi']w_1, \ldots, [p_h, \phi']w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w'_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$;
7) if $\phi = EX^a\phi'$, then $[p, EX^a\phi']\gamma \xhookrightarrow{[(\rho,\rho')]} \{p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$ and $p'' = [p', \phi']$, or $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$, $p'' = [p', \phi]^r$, and $w = \gamma_1\gamma_2^r$;
8) if $\phi = EX^a\phi_1$, $\phi = E[\phi_1U^a\phi_2]$, or $\phi = E[\phi_1R^a\phi_2]$, then $[p, \phi]^r\gamma \xhookrightarrow{[(\rho,\rho')]} \{[p', \phi]^rw\} \in \Delta'$ for each $p\gamma \xrightarrow{(\rho,\rho')}_t p'w \in \Delta$;
9) if $\phi = AX^a\phi'$, then $[p, AX^a\phi']\gamma \xhookrightarrow{[\sigma_1,\ldots,\sigma_h]} \{p'_1w_1, \ldots, p'_hw_h\} \in \Delta'$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w'_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then, $p'_i = [p_i, \phi']$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = [p_i, True]$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $[p_i, \phi']^r$ and $w_i = \gamma_1\gamma_2^r$;
10) if $\phi = AX^a\phi_1$, $\phi = A[\phi_1U^a\phi_2]$, or $\phi = A[\phi_1R^a\phi_2]$, then $[p, \phi]^r\gamma \xhookrightarrow{[\sigma_1,\ldots,\sigma_h]} \{[p_1, \phi]^rw_1, \ldots, [p_h, \phi]^rw_h\} \in \Delta'$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_t p_1w_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_t p_hw_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, where for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$;
11) if $\phi = QX^a\phi_1$ for $Q \in \{E, A\}$, then $[p, \phi]^r\gamma^r \xhookrightarrow{[\top]} \{[p, \phi_1]\gamma\} \in \Delta'$;
12) if $\phi = Q[\phi_1U^b\phi_2]$ for $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, then $[p, \phi]\gamma \xhookrightarrow{[\top]} \{[p, \phi_2]\gamma\}$;
13) if $\phi = Q[\phi_1R^b\phi_2]$ for $Q \in \{E, A\}$ and $b \in \{g, a, c\}$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$;
14) if $\phi = E[\phi_1U^g\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[(\rho,\rho),(\rho,\rho')]} \{[p, \phi_1]\gamma, [p', \phi]w\}$, s.t. either $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$, $p\gamma \xrightarrow{(\rho,\rho')}_{ret} p'w \in \Delta$, or $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$ and $w = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$;
15) if $\phi = A[\phi_1U^g\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\sigma_1,\ldots,\sigma_h]} \{[p, \phi_1]\gamma, [p_1, \phi]w_1, \ldots, [p_h, \phi]w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w'_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$;
16) if $\phi = E[\phi_1U^a\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[(\rho,\rho),(\rho,\rho')]} \{[p, \phi_1]\gamma, p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$ and $p'' = [p', \phi]$, or $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$, $p'' = [p', \phi]^r$, and $w = \gamma_1\gamma_2^r$;
17) if $\phi = A[\phi_1U^a\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\sigma_1,\ldots,\sigma_h]} \{[p, \phi_1]\gamma, p'_1w_1, \ldots, p'_hw_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w'_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then $p'_i = [p_i, \phi]$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = [p_i, False]$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_1\gamma_2^r$;
18) if $\phi = Q[\phi_1U^a\phi_2]$ or $\phi = Q[\phi_1R^a\phi_2]$ for $Q \in \{E, A\}$, then $[p, \phi]^r\gamma^r \xhookrightarrow{[\top]} \{[p, \phi]\gamma\} \in \Delta'$;
19) if $\phi = E[\phi_1R^g\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[(\rho,\rho),(\rho,\rho')]} \{[p, \phi_2]\gamma, [p, \phi]w\}$, s.t. either $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$, $p\gamma \xrightarrow{(\rho,\rho')}_{ret} p'w \in \Delta$, or $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$ and $w = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$;
20) if $\phi = A[\phi_1R^g\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\sigma_1,\ldots,\sigma_h]} \{[p, \phi_2]\gamma, [p_1, \phi]w_1, \ldots, [p_h, \phi]w_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w'_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$, if $t_i \in \{int, ret\}$, then $w_i = w'_i$, and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $w_i = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$;
21) if $\phi = E[\phi_1R^a\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[(\rho,\rho),(\rho,\rho')]} \{[p, \phi_2]\gamma, p''w\}$, s.t. either $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$ and $p'' = [p', \phi]$, or $p\gamma \xrightarrow{(\rho,\rho')}_{ret} p'w \in \Delta$ and $p'' = [p', True]$, or $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2 \in \Delta$, $p'' = [p', \phi]^r$, and $w = \gamma_1\gamma_2^r$;
22) if $\phi = A[\phi_1R^a\phi_2]$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\sigma_1,\ldots,\sigma_h]} \{[p, \phi_2]\gamma, p'_1w_1, \ldots, p'_hw_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w'_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw'_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, and for $1 \le i \le h$,

$\sigma_i = (\rho_i, \rho'_i)$; if $t_i = int$, then, $p'_i = [p_i, \phi]$ and $w_i = w'_i$; if $t_i = ret$, then $p'_i = [p_i, True]$ and $w_i = w'_i = \varepsilon$; and if $t_i = call$ and $w'_i = \gamma_1\gamma_2$, then $p'_i = [p_i, \phi]^r$ and $w_i = \gamma_1\gamma_2^r$;

23) if $\phi = QX^c\phi'$, where $Q \in \{E, A\}$, then $[p, \phi]\gamma \xhookrightarrow{[\top]} \{[p^c, \phi]\varepsilon\}$;
24) if $\phi = Q[\phi_1 U^c \phi_2]$, where $Q \in \{E, A\}$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\top]} \{[p, \phi_1]\gamma, [p^c, \phi]\varepsilon\}$;
25) if $\phi = Q[\phi_1 R^c \phi_2]$, where $Q \in \{E, A\}$, then $[p, \phi]\gamma \xhookrightarrow{[\top,\top]} \{[p, \phi_2]\gamma, [p^c, \phi]\varepsilon\}$;
26) $[p^c, \phi]\gamma \xhookrightarrow{[\top]} \{[p^c, \phi]\varepsilon\} \in \Delta'$ for $\gamma \neq \#$ and $[p^c, \phi]\# \xhookrightarrow{[\top]} \{[p^c, \phi]\#\}$;
27) if $\phi = QX^c\phi_1$ for $Q \in \{E, A\}$, $[p^c, \phi]\lfloor\gamma, p_0, \gamma_0\rfloor \xhookrightarrow{restore} [p_0, \phi_1]\gamma_0 \in restore$ for $p_0 \in P$ and $\gamma_0 \in \Gamma$;
28) if $\phi = Q[\phi_1 U^c \phi_2]$ or $\phi = Q[\phi_1 R^c \phi_2]$ for $Q \in \{E, A\}$, $[p^c, \phi]\lfloor\gamma, p_0, \gamma_0\rfloor \xhookrightarrow{restore} [p_0, \phi]\gamma_0 \in restore$ for $p_0 \in P$ and $\gamma_0 \in \Gamma$;
29) $[p, \phi]\lfloor\gamma, p_0, \gamma_0\rfloor \xhookrightarrow{discard} [p, \phi]\gamma \in discard$ for $p \neq p^c$, $p_0 \in P$, and $\gamma_0 \in \Gamma$.

We can show that (the proof can be found in Appendix A):

Theorem 1. *For an SM-PDS $\mathcal{P}$ and a BranchCARET formula ψ, we can compute an SM-ABPDS $\mathcal{BP} = (P', \Gamma', X, \Delta', restore, discard, \Delta, F)$, s.t. for any configuration $(\langle p, w\rangle, \theta)$ of $\mathcal{P}$, $(\langle p, w\rangle, \theta) \models \psi$ iff there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], w\rangle, \theta)$.*

Roughly speaking, $\mathcal{BP}$ is a kind of product between $\mathcal{P}$ and ψ. We construct $\mathcal{BP}$ in such a way that for a configuration $(\langle p, w\rangle, \theta)$ of $\mathcal{P}$, $(\langle p, w\rangle, \theta) \models \psi$ iff there exists an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], w\rangle, \theta)$. Suppose that $(\langle p, \gamma\omega\rangle, \theta) \models \psi$, i.e. there should be an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], \gamma\omega\rangle, \theta)$.

If $\psi = a \in AP$, then our construction (item 1) creates a rule of the form $[p, a]\gamma \xhookrightarrow{[\top]} \{[p, a]\gamma\}$ only if $p \in \lambda(a)$, which creates an infinite loop of the form $(\langle [p, a], \gamma\omega\rangle, \theta) \Longrightarrow (\langle [p, a], \gamma\omega\rangle, \theta) \Longrightarrow \ldots$. To ensure that this infinite loop constitutes an accepting run, we add $[p, a]$ to the set of accepting states F. The intuition behind $\psi = \neg a$ is similar.

If $\psi = \phi_1 \vee \phi_2$, then either $(\langle p, \gamma\omega\rangle, \theta) \models \phi_1$, or $(\langle p, \gamma\omega\rangle, \theta) \models \phi_2$. Our construction (item 3) creates two rules of the forms $[p, \psi]\gamma \xhookrightarrow{[\top]} \{[p, \phi_1]\gamma\}$ and $[p, \psi]\gamma \xhookrightarrow{[\top]} \{[p, \phi_2]\gamma\}$. Therefore, there is an accepting run from $(\langle [p, \psi], \gamma\omega\rangle, \theta)$ only if there is an accepting run from either $(\langle [p, \phi_1], \gamma\omega\rangle, \theta)$ or $(\langle [p, \phi_2], \gamma\omega\rangle, \theta)$.

If $\psi = \phi_1 \wedge \phi_2$, then $(\langle p, \gamma\omega\rangle, \theta) \models \phi_1$ and $(\langle p, \gamma\omega\rangle, \theta) \models \phi_2$. Our construction (item 4) creates a rule of the form $[p, \psi]\gamma \xhookrightarrow{[\top]} \{[p, \phi_1]\gamma, [p, \phi_2]\gamma\}$. Therefore, there is an accepting run from $(\langle [p, \psi], \gamma\omega\rangle, \theta)$ only if there are accepting runs from both $(\langle [p, \phi_1], \gamma\omega\rangle, \theta)$ and $(\langle [p, \phi_2], \gamma\omega\rangle, \theta)$.

If $\psi = EX^g\phi_1$, then this formula is satisfied only if there is a global successor $(\langle p', v\omega\rangle, \theta')$ in $\mathcal{P}$ satisfying ϕ_1. This successor is necessarily obtained from a rule of the form $\langle p, \gamma\rangle \xrightarrow{(\rho,\rho')}_t \langle p', v\rangle \in \Delta$, s.t. $\theta' = (\theta \backslash \rho) \cup \rho'$. Our construction (item 5) then creates a rule in $\mathcal{BP}$ depending on t (type of the corresponding transition in $\mathcal{P}$):

- If $t = int$ or $t = ret$, then our construction (item 5) creates a rule of the form $[p, \psi]\gamma \xhookrightarrow{[(\rho,\rho')]} \{[p', \phi_1]v\}$. Therefore, there is an accepting run in $\mathcal{BP}$ starting from $(\langle [p, \psi], \gamma\omega\rangle, \theta)$ only if there is an accepting run from $(\langle [p', \phi_1], v\omega\rangle, \theta')$, which corresponds to checking if the global successor $(\langle p', v\omega\rangle, \theta')$ satisfies ϕ_1;
- If $t = call$, then the applied rule has the form $\langle p, \gamma\rangle \xrightarrow{(\rho,\rho')}_{call} \langle p', \gamma_1\gamma_2\rangle$. Our construction (item 5) creates a rule of the form $[p, \psi]\gamma \xhookrightarrow{[(\rho,\rho')]} \{[p', \phi_1]\gamma_1\lfloor\gamma_2, p, \gamma\rfloor\}$, where $\lfloor\gamma_2, p, \gamma\rfloor \in X$ is a checkpoint. Therefore, in $\mathcal{BP}$, there is a transition $(\langle [p, \psi], \gamma\omega\rangle, \theta) \Longrightarrow (\langle [p', \phi_1], \gamma_1\lfloor\gamma_2, p, \gamma\rfloor^\theta\omega\rangle, \theta')$. Intuitively, $\lfloor\gamma_2, p, \gamma\rfloor^\theta$ allows to record that the call of the return point γ_2 was performed at control location p, with γ on the top of the stack, and with phase θ. There are two cases depending on whether in the run, we need to check the caller successor (this checkpoint is needed for this case), or not:

1) If we reach the corresponding return of this call without hitting any caller successor, then, in this case, there was no need to use this checkpoint. Thus, we discard the checkpoint using the discard rule of the form $[p'', \phi']\lfloor\gamma_2, p, \gamma\rfloor \xhookrightarrow{discard} [p'', \phi']\gamma_2$ that will simply discard the useless (in this case) information (p, γ, and θ) and keeps only the needed information (the return location γ_2).
2) Suppose, in run π on $\mathcal{P}$, before reaching the corresponding return, ϕ_1 requires to check some configuration $(\langle p'', \omega''\rangle, \theta'')$ against some formula $EX^c\phi'$, i.e. one of children on the path is a configuration of the form $(\langle [p'', EX^c\phi'], \omega''\rangle, \theta'')$. In this case, we need to come back to the caller site $(\langle p, \gamma\omega\rangle, \theta)$ and check whether it satisfies ϕ'. This is where the checkpoint $\lfloor\gamma_2, p, \gamma\rfloor^\theta$ is needed. It allows $\mathcal{BP}$ to come back (and restore) the caller site $(\langle p, \gamma\omega\rangle, \theta)$. This is done as follows. Since the corresponding return has not been reached, ω'' has the form $u\gamma_2\omega$ (it contains the return location γ_2). Thus, in $\mathcal{BP}$ the corresponding configuration is $(\langle [p'', EX^c\phi'], u\lfloor\gamma_2, p, \gamma\rfloor^\theta\omega\rangle, \theta'')$. Our construction (item 23) creates a rule of the form $[p'', EX^c\phi']\gamma' \xhookrightarrow{[\top]} \{[p^c, EX^c\phi']\varepsilon\}$ for every $\gamma' \in \Gamma$, where p^c is a new control location. Then, our construction (item 26) creates a rule of the form $[p^c, EX^c\phi']\gamma' \xhookrightarrow{[\top]} \{[p^c, EX^c\phi']\varepsilon\}$ for every $\gamma' \in \Gamma$, i.e. these two rules pop every symbol that is not a checkpoint from the stack and transition into a control location $[p^c, EX^c\phi']$. Then, our construction (item 27) creates a rule of

the form $[p^c, EX^c\phi']\lfloor\gamma_2, p, \gamma\rfloor \xhookrightarrow{restore} [p, \phi']\gamma$. Therefore, there is a transition of the form $(\langle[p^c, EX^c\phi'], \lfloor\gamma_2, p, \gamma\rfloor^\theta\omega\rangle, \theta'') \Longrightarrow (\langle[p, \phi'], \gamma\omega\rangle, \theta)$. Indeed, there is an accepting run from $(\langle[p'', EX^c\phi'], u\lfloor\gamma_2, p, \gamma\rfloor^\theta\omega\rangle, \theta)$ only if the caller successor $(\langle p, \gamma\omega\rangle, \theta)$ satisfies ϕ'.

Therefore, there is an accepting run in $\mathcal{BP}$ starting from $(\langle[p, \psi], \gamma\omega\rangle, \theta)$ only if there is an accepting run from $(\langle[p', \phi_1], \gamma_1\lfloor\gamma_2, p, \gamma\rfloor^\theta\omega\rangle, \theta')$, which corresponds to checking if the global successor $(\langle p', \gamma_1\gamma_2\omega\rangle, \theta')$ satisfies ϕ_1;

If $\psi = AX^g\phi_1$, then every global successor of $(\langle p, \gamma\omega\rangle, \theta)$ has to satisfy ϕ_1. Let then $\{\langle p, \gamma\rangle \xhookrightarrow{(\rho_1, \rho_1')}_{t_1} \langle p_1, v_1\rangle, \ldots, \langle p, \gamma\rangle \xhookrightarrow{(\rho_h, \rho_h')}_{t_h} \langle p_h, v_h\rangle\}$ be the set of all rules in Δ with $\langle p, \gamma\rangle$ on the left hand side. To check every possible successor, we need to use alternation: our construction (item 6) creates a rule of the form $[p, \psi]\gamma \xhookrightarrow{[\sigma_1, \ldots, \sigma_h]} \{[p_1, \phi_1]v_1', \ldots, [p_h, \phi_1]v_h'\}$, where for $1 \leq i \leq h$, $\sigma_i = (\rho_i, \rho_i')$, and v_i' depends on the transition type t_i as previously:

- If $t = int$ or $t = ret$, then $v_i' = v_i$,
- If $t = call$, then v_i has the form $v_i = \gamma_1\gamma_2$, and $v_i' = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$.

This expresses that $\mathcal{BP}$ will simultaneously transition into the configurations of the form $(\langle[p_i, \phi_1], v_i'\omega\rangle, \theta_i)$, where for $1 \leq i \leq h$, $(\langle p_i, v_i\omega\rangle, \theta_i)$ is a potential global successor of $(\langle p, \gamma\omega\rangle, \theta)$.

If $\psi = EX^a\phi_1$, then we need to check the abstract successor of $(\langle p, \gamma\omega\rangle, \theta)$. The abstract successor depends on the type of rule that can be applied:

- If $\mathcal{P}$ applies a rule of the form $\langle p, \gamma\rangle \xhookrightarrow{(\rho, \rho')}_{int} \langle p', v\rangle \in \Delta$, then the abstract successor is the next configuration $\big(\langle p', v\omega\rangle, (\theta\backslash\rho) \cup \rho'\big)$. Hence, our construction (item 7) creates a rule of the form $[p, EX^a\phi_1]\gamma \xhookrightarrow{[(\rho, \rho')]} \{[p', \phi_1]v\}$ that ensures that $\mathcal{BP}$ contains an accepting run starting from $(\langle[p, EX^a\phi_1], \gamma\omega\rangle, \theta)$ if there is an accepting run from $\big(\langle[p', \phi_1], v\omega\rangle, (\theta\backslash\rho) \cup \rho'\big)$.
- If $\mathcal{P}$ applies a rule of the form $\langle p, \gamma\rangle \xhookrightarrow{(\rho, \rho')}_{call} \langle p', \gamma_1\gamma_2\rangle \in \Delta$, then the abstract successor is the corresponding return location. Suppose the corresponding return in $\mathcal{P}$ is obtained from the path of the form $(\langle p, \gamma\omega\rangle, \theta) \overset{call}{\Longrightarrow} (\langle p', \gamma_1\gamma_2\rangle, \theta_0) \Longrightarrow \ldots \Longrightarrow (\langle p_k, \gamma_k\gamma_2\omega\rangle, \theta_k) \overset{ret}{\Longrightarrow} (\langle p'', \gamma_2\omega\rangle, \theta')$, where γ_2 is the corresponding return location. Our construction (item 7) creates a rule of the form $[p, \psi]\gamma \xhookrightarrow{[(\rho, \rho')]} \{[p', \psi]^r\gamma_1\gamma_2^r\}$, i.e. $\mathcal{BP}$ will have a transition $(\langle[p, \psi], \gamma\omega\rangle, \theta) \Longrightarrow \{(\langle[p', \psi]^r, \gamma_1\gamma_2^r\omega\rangle, \theta_0)\}$. The symbol γ_2^r expresses that when it is located on top of the stack, then the corresponding return has been reached. The control location $[p', \psi]^r$ expresses that $\mathcal{BP}$ searches for the corresponding return. Then, to handle control locations of the form $[p, \psi]^r$, our construction (item 8) creates rules of the form $[p_1, \psi]^r\gamma' \xhookrightarrow{[(\rho_1, \rho_1')]} \{[p_2, \psi]^r v_1\}$ for each rule of the form $p_1\gamma' \xhookrightarrow{(\rho_1, \rho_1')}_t p_2v_1 \in \Delta$ (for any p_1, γ', ρ_1, ρ_1', t, p_2, and v_1). In other words, $\mathcal{BP}$ mirrors the execution of $\mathcal{P}$ when at control locations of the form $[p_1, \psi]^r$. Then, our construction (item 11) creates a rule of the form $[p'', \psi]^r\gamma_2^r \xhookrightarrow{[\top]} \{[p'', \phi_1]\gamma_2\}$, such that $(\langle[p'', \psi]^r, \gamma_2^r\omega\rangle, \theta') \Longrightarrow \{(\langle[p'', \phi_1], \gamma_2\omega\rangle, \theta')\}$. This expresses that when $\mathcal{BP}$ is at a control location of the form $[p, \psi]^r$ (labelled with r), then $\mathcal{BP}$ is in a special 'mode' that searches for the corresponding return of the call. The corresponding return is reached only when the return location γ_2 is on the top of the stack. We mark the return location γ_2^r with r to specify that when γ_2^r is on the top of the stack, then $\mathcal{BP}$ needs to stop looking for the corresponding return and check the current configuration (i.e. the abstract successor) against ϕ_1. Hence, there is an accepting run from $(\langle[p, EX^a\phi_1], \gamma\omega\rangle, \theta)$ in $\mathcal{BP}$ if there is an accepting run from $(\langle[p'', \phi_1], \gamma_2\omega\rangle, \theta')$ in $\mathcal{BP}$, which corresponds to the abstract successor $(\langle p'', \gamma_2\omega\rangle, \theta')$ in $\mathcal{P}$ satisfying ϕ_1.
- If $\mathcal{P}$ applies a ret rule, then there is no abstract successor and $EX^a\phi_1$ is not satisfied. Hence, our construction (item 7) does not construct any rules.

If $\psi = AX^a\phi_1$, then $\mathcal{BP}$ must check every possible abstract successor. Suppose $\{p\gamma \xhookrightarrow{\sigma_1}_{t_1} p_1w_1, \ldots, p\gamma \xhookrightarrow{\sigma_h}_{t_h} p_hw_h\}$ are all rules in Δ with $p\gamma$ on the left-hand side. Then, our construction (item 9) creates a rule of the form $[p, AX^a\phi_1]\gamma \xhookrightarrow{[\sigma_1, \ldots, \sigma_h]} \{p_1'w_1', \ldots, p_h'w_h'\}$. For each $1 \leq i \leq h$, p_i' and w_i' depend on the type t_i of the corresponding rule in $\mathcal{P}$:

- If $t_i = int$, then the intuition is similar to the case of $AX^g\phi_1$.
- If $t_i = call$ and $w_i = \gamma_i\gamma_i'$, then $p_i' = [p_i, \phi]^r$ and $w_i' = \gamma_i\gamma_i'^r$, i.e. the i-th child of $(\langle[p, AX^a\phi_1], \gamma\omega\rangle, \theta)$ has the form $(\langle[p_i, \phi]^r, \gamma_i\gamma_i'^r\omega\rangle, \theta_i)$, where $\theta_i = (\theta\backslash\rho_i) \cup \rho_i'$. In contrast to EX^a, $\mathcal{BP}$ needs to find all possible corresponding returns and cases when there is no corresponding return. Intuitively, our construction (item 10) creates such rules that constitute a tree η, where each branch of η starts from $(\langle[p_i, \phi]^r, \gamma_i\gamma_i'^r\omega\rangle, \theta)$ and corresponds to one valid run π in $\mathcal{P}$ until the corresponding return. Let us take one such branch η_π. There are two cases:
  - π has a corresponding return $c \overset{ret}{\Longrightarrow} (\langle p'', \gamma_2'\omega\rangle, \theta')$ for some $c \in Conf$. Then, rules created by our construction (item 10) ensure that η_π will reach $(\langle[p'', \phi]^r, \gamma_2'^r\omega\rangle, \theta')$, and rules of the form $[p'', \psi]^r\gamma_2^r \xhookrightarrow{[\top]} \{[p'', \phi_1]\gamma_2\}$ described in the previous case, allow the transition $(\langle[p'', \phi]^r, \gamma_2'^r\omega\rangle, \theta') \Longrightarrow (\langle[p'', \phi'], \gamma_2'\omega\rangle, \theta')$. Hence, η_π is accepting only if $\mathcal{BP}$ contains an accepting run from $(\langle[p'', \phi_1], \gamma_2'\omega\rangle, \theta')$, which corresponds to the abstract successor $(\langle p'', \gamma_2'\omega\rangle, \theta')$ in $\mathcal{P}$;

– π does not have a corresponding return, i.e. it contains an infinite loop before reaching a corresponding return. Then, η_π will never contain γ_2^r on top of the stack. By the construction of $\mathcal{BP}$, the control locations of the form $[p,\psi]^r$ can 'remove' the mark r only using the above mentioned rule. Hence, every control location on η_π has the form $[p_k,\psi]^r$ for any $p_k \in P$. Since $[p_k, AX^a\phi_1]^r \in F^a \subseteq F$, then η_π is accepting.

- If $t_i = ret$, then there is no abstract successor, and $p'_i = [p_i, True]$, i.e. there is always an accepting run. Such branch is required to cover the case when there are only ret rules that can be applied. In this case, there are no abstract successors, thus, it satisfies $AX^a\phi_1$, and therefore, this run needs to be accepting.

Therefore, there is an accepting run from $(\langle[p, AX^a\phi_1], \gamma\omega\rangle, \theta)$ only if for each abstract successor $(\langle p_i, w_i\omega\rangle, \theta_i)$ in $\mathcal{P}$, there is an accepting run in $\mathcal{BP}$ starting from $(\langle[p_i, \phi_1], w_i\omega\rangle, \theta_i)$.

If $\psi = E[\phi_1 U^g \phi_2]$, then either $(\langle p, \gamma\omega\rangle, \theta) \models \phi_2$, or $(\langle p, \gamma\omega\rangle, \theta) \models \phi_1$ and there is a global successor $(\langle p', v\omega\rangle, \theta')$, s.t. $(\langle p', v\omega\rangle, \theta') \models \psi$. Thus, our construction creates rules for these two cases:

1) $[p,\psi]\gamma \xhookrightarrow{[\top]} \{[p,\phi_2]\gamma\}$. This rule ensures that there is an accepting run from $(\langle[p,\psi],\gamma\omega\rangle,\theta)$ if there is an accepting run from $(\langle[p,\phi_2],\gamma\omega\rangle,\theta)$.
2) let $\langle p,\gamma\rangle \xhookrightarrow{(\rho,\rho')}_t \langle p', v\rangle$ be the rule from which the successor is obtained. Then, our construction (item 14) creates a rule of the form $[p,\psi]\gamma \xhookrightarrow{[(\rho,\rho),(\rho,\rho')]} \{[p,\phi_1]\gamma, [p',\psi]v'\}$, where v' depends on the transition type t as previously:
   - If $t = int$ or $t = ret$, then $v' = v$,
   - If $t = call$, then v has the form $v = \gamma_1\gamma_2$, and $v' = \gamma_1\lfloor\gamma_2, p, \gamma\rfloor$.

   Thus, there is an accepting run from $(\langle[p,\psi],\gamma\omega\rangle,\theta)$ if there are accepting runs from $(\langle[p,\phi_1],\gamma\omega\rangle,\theta)$ and from $(\langle[p',\psi], v'\omega\rangle, \theta')$, which corresponds to the global successor.

If $\psi = A[\phi_1 U^g \phi_2]$, the intuition is similar to $E[\phi_1 U^g \phi_2]$.

If $\psi = E[\phi_1 U^a \phi_2]$. For a run π on $\mathcal{P}$, suppose that $\pi(i) = (\langle p, \gamma\omega\rangle, \theta)$ and $(\pi, i) \models \psi$. Then either $(\pi, i) \models \phi_2$, or $(\pi, next^a_\pi(i)) \models E[\phi_1 U^a \phi_2]$ and $(\pi, i) \models \phi_1$. Intuition behind checking $(\pi, i) \models \phi_2$ is the same as in $\psi = E[\phi_1 U^g \phi_2]$. If $(\pi, next^a_\pi(i)) \models E[\phi_1 U^a \phi_2]$ and $(\pi, i) \models \phi_1$, our construction (item 16) creates a rule depending on the type of rule that was applied at (π, i):

- If the rule has the form $p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w$, then the intuition is the same as in $E[\phi_1 U^g \phi_2]$;
- If the rule has the form $p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2$, then our construction (item 16) creates a rule of the form $[p,\psi]\gamma \xhookrightarrow{[(\rho,\rho),(\rho,\rho')]} \{[p,\phi_1]\gamma, [p',\psi]^r\gamma_1\gamma_2^r\}$, thus allowing the transition $(\langle[p,\psi],\gamma\omega\rangle,\theta) \Longrightarrow \{(\langle[p,\phi_1],\gamma\omega\rangle,\theta), (\langle[p',\psi]^r, \gamma_1\gamma_2^r\omega\rangle, \theta')\}$, where $\theta' = (\theta\backslash\rho) \cup \rho'$. There are two cases depending on whether the abstract successor of $\pi(i)$ exists:
  - Suppose the abstract successor of $\pi(i)$ exists and $\pi(next^a_\pi(i)) = (\langle p'', \gamma_2\omega\rangle, \theta')$. Similar to the case of $EX^a\phi_1$, $(\langle[p',\psi]^r\gamma_1\gamma_2^r\omega\rangle, \theta)$ will reach the configuration $(\langle[p'',\psi],\gamma_2\rangle,\theta')$, which corresponds to the abstract successor $\pi(next^a_\pi(i))$. Hence there is an accepting run in $\mathcal{BP}$ starting from $(\langle[p,\psi],\gamma\omega\rangle,\theta)$ if there are accepting runs from both $(\langle[p,\phi_1],\gamma\omega\rangle,\theta)$ and $(\langle[p'',\psi],\gamma_2\omega\rangle,\theta')$, i.e. if $(\pi, next^a_\pi(i)) \models \psi$ and $(\pi, i) \models \phi_1$. Moreover, the condition that ϕ_2 is satisfied in the future is guaranteed by the fact that $[p, E[\phi_1 U^a \phi_2]] \notin F$ for any $p \in P$.
  - If the abstract successor $next^a_\pi(i)$ is undefined, i.e. there is no corresponding return. Unlike with $AX^a\phi_1$, (π, i) should not satisfy ψ, because ϕ_2 is never satisfied on the abstract path on π. In this case, $\mathcal{BP}$ will visit only control locations of the form $[p_k,\psi]^r$ after $\pi(i)$. This run is not accepting because $[p_k,\psi]^r \notin F$.
- If the rule has the form $p\gamma \xhookrightarrow{\sigma}_{ret} p'w$, then there is no abstract successor, so our construction (item 16) does not construct the corresponding rule and hence, there is no accepting run from $(\langle[p,\psi],\gamma\omega\rangle,\theta)$.

If $\psi = A[\phi_1 U^a \phi_2]$. For a run π on $\mathcal{P}$, suppose that $\pi(i) = (\langle p,\gamma\omega\rangle,\theta)$ and $(\pi,i) \models \psi$. Then our construction (item 17) creates a rule of the form $[p,\psi]\gamma \xhookrightarrow{[\top,\sigma_1,\ldots,\sigma_h]} \{[p,\phi_1]\gamma, p'_1w'_1, \ldots, p'_hw'_h\}$, s.t. $\{p\gamma \xrightarrow{(\rho_1,\rho'_1)}_{t_1} p_1w_1, \ldots, p\gamma \xrightarrow{(\rho_h,\rho'_h)}_{t_h} p_hw_h\}$ is the set of rules in Δ with $p\gamma$ as left-hand side, for $1 \le i \le h$, $\sigma_i = (\rho_i, \rho'_i)$, and p'_i and w'_i depend on t_i:

- If $t_i = int$, then $p'_i = [p_i,\psi]$ and $w'_i = w_i$.
- If $t_i = call$ and $w_i = \gamma_i\gamma'_i$, then $p'_i = [p_i,\phi]^r$ and $w'_i = \gamma_i\gamma_i'^r$.
- If $t_i = ret$, then $p'_i = [p_i, False]$ because there is no abstract successor. Unlike with $AX^a\phi_1$, the absence of the abstract successor does not imply the absence of the abstract path. In fact, the abstract path starting from i is a sequence $s = i$, which contains a single element i. Therefore, ϕ_2 must be satisfied on s. However, since s is a single element i, then (π', i) must satisfy ϕ_2, i.e. (π, i) must satisfy ϕ_2. In this case, the constructed rule should not be applied. To ensure this, p'_i is $[p_i, False]$, i.e. $\mathcal{BP}$ always rejects runs that can perform $return$ and try to check the abstract successors. Instead, the accepting run must be found by applying a rule checking that $(\pi, i) \models \phi_2$.

If $\psi = E[\phi_1 R^g \phi_2]$. For a run π on $\mathcal{P}$, suppose that $\pi(i) = (\langle p,\gamma\omega\rangle,\theta)$ and $(\pi,i) \models \psi$. There are two cases: either there is $k \ge i$, s.t. $(\pi,k) \models \phi_1$, and for each $i \le j \le k$ $(\pi,j) \models \phi_2$, or for all $k \ge i$, $(\pi,k) \models \phi_2$. In the first case, the intuition is similar to the case of $E[\phi_1 U^g \phi_2]$. In the latter case, our construction ensures that the run in $\mathcal{BP}$ is accepting by the definition of F, i.e. that $[p, E[\phi_1 R^g \phi_2]] \in F$ for any $p \in P'$.

If $\psi = A[\phi_1 R^g \phi_2]$, the intuition is similar to $E[\phi_1 R^g \phi_2]$.

If $\psi = E[\phi_1 R^a \phi_2]$, the intuition is similar to $E[\phi_1 U^g \phi_2]$.

If $\psi = A[\phi_1 R^a \phi_2]$, the intuition is similar to $E[\phi_1 R^a \phi_2]$.

If $\psi = E[\phi_1 U^c \phi_2]$. Suppose that $\pi(i) = (\langle p, \gamma\omega\rangle, \theta)$ and $(\pi, i) \models \psi$. Then either $(\pi, i) \models \phi_2$, or $(\pi, i) \models \phi_1$ and $(\pi, next^c_\pi(i)) \models \psi$. The first case is similar to $E[\phi_1 U^g \phi_2]$. In the second case, our construction (item 24) creates a rule of the form $[p, E[\phi_1 U^c \phi_2]]\gamma \xhookrightarrow{[\top,\top]} \{[p, \phi_1]\gamma, [p^c, E[\phi_1 U^c \phi_2]]\varepsilon\}$. Similar to the case of $EX^c\phi$, there is a transition $(\langle [p^c, \psi], \omega\rangle, \theta) \Longrightarrow^* \{(\langle [p', \psi], w'\rangle, \theta')\}$, where $\pi(next^c_\pi(i)) = (\langle p', w'\rangle, \theta')$. Therefore, there is an accepting run from $(\langle [p, \psi], \gamma\omega\rangle, \theta)$ if there are accepting runs from both $(\langle [p, \phi_1], \gamma\omega\rangle, \theta)$ and $(\langle [p', \psi], w'\rangle, \theta')$, i.e. if $(\langle p, \gamma\omega\rangle, \theta)$ satisfies ϕ_1 and the caller successor $(\langle p', w'\rangle, \theta')$ satisfies ψ. The condition that ϕ_2 must be satisfied on the caller path is ensured by the definition of F. Suppose for some k, $next^c_\pi(k) = \bot$ and $\pi(k) = (\langle p_k, w_k\#\rangle, \theta_k)$. In this case, the corresponding configuration in $\mathcal{BP}$ is of the form $(\langle [p_k, \phi], w_k\#\rangle, \theta_k)$, s.t. there are no checkpoints in w_k (because there are no prior calls that pushed a checkpoint). First, $\mathcal{BP}$ will pop w_k from the stack, obtaining the configuration $(\langle [p^c, \phi], \#\rangle, \theta_k)$. Then, our construction creates a rule (from Item 26) of the form $[p^c, \psi]\# \xhookrightarrow{[\top]} \{[p^c, \psi]\#\}$ that loops over the control location $[p^c, \psi]$. And since $[p^c, \psi] \notin F$, then this run is not accepting. The same intuition is applied for $A[\phi_1 U^c \phi_2]$ because (π, i) can have at most one caller successor.

If $\psi = E[\phi_1 R^c \phi_2]$. The intuition is similar to $E[\phi_1 U^c \phi_2]$. The only difference is that $\psi = E[\phi_1 R^c \phi_2]$ allows ϕ_2 to hold for every caller successor without ϕ_1 being satisfied. Our construction ensures that it holds for $\mathcal{BP}$ by the definition of F: $[p^c, \psi] \in F^c \subseteq F$, therefore, such run is accepting. A similar intuition is applied for the cases $\phi = AX^c\phi_1$ and $\phi = A[\phi_1 R^c \phi_2]$.

C. Solving the Emptiness Problem of SM-ABPDS with Checkpoints

To find accepting runs in $\mathcal{BP}$, we need to define functions $pre^+, pre^* : 2^{Conf_{\mathcal{BP}}} \to 2^{Conf_{\mathcal{BP}}}$, which compute the transitive and reflexive-transitive sets of predecessors respectively. The functions are defined as $pre^*(W) = \{c \mid c \in W \text{ or } c \Longrightarrow S, \text{ s.t. } S \subseteq pre^*(W)\}$, and $pre^+(W) = \{c \mid c \Longrightarrow S, \text{ s.t. } S \subseteq pre^*(W)\}$. Let $Y_0 = Conf_{\mathcal{BP}}$ be the set of configurations from which there is a run that visits accepting control locations at least 0 times. Then, for $j > 0$, let $Y_j = pre^+\big(Y_{j-1} \cap (F \times G^* \times 2^\delta)\big)$ be the set of configurations from which there is a run that visits accepting control locations at least j times. Then, we can find the set of configurations $Y_{\mathcal{BP}}$, from which there is a run that visits accepting control locations infinitely many times by defining $Y_{\mathcal{BP}} = \bigcap_{j=0} Y_j$. We can show that (the proof can be found in Appendix B):

Theorem 2. *For any configuration c of $\mathcal{BP}$, there is an accepting run starting from c iff $c \in Y_{\mathcal{BP}}$.*

Intuitively, suppose there is an accepting run η from c in $\mathcal{BP}$. By definition, $c \in Y_0$. Then, each branch of η must visit an accepting control location at least once. Suppose for the i-th branch, it visits the first accepting control location at node c_i, i.e. $c_i \in F \times G^* \times 2^\delta$, by definition of Y_0, $c_i \in Y_0$ and hence, $c_i \in Y_0 \cap F \times G^* \times 2^\delta$. Moreover, since c reaches $\bigcup_i c_i$ in η, then $c \in pre^+(\bigcup_i c_i)$, and thus, $c \in Y_1$. Since each branch of η must visit accepting control locations at least twice, then for each c_i, there must be a run starting from c_i that visits accepting control locations at least once. Inductively, we get that $c_i \in Y_1$. And therefore, $c \in Y_2$ (because $c_i \in F \times G^* \times 2^\delta$ and $c \in pre^+(\bigcup_i c_i)$). Then, since each branch of η must visit accepting control locations infinitely many times, we can inductively apply this reasoning for each $j \geq 0$, obtaining that $c \in Y_j$. And thus, since $Y_{\mathcal{BP}} = \bigcap_{j\geq 0} Y_j$, we get that $c \in Y_{\mathcal{BP}}$.

D. Computing $Y_{\mathcal{BP}}$

We show that we can efficiently compute $Y_{\mathcal{BP}}$ by constructing an alternating finite automaton $\mathcal{A}$.

Definition 4. *An Alternating Multi Automaton (AMA) is a tuple $A = (Q, G, T, I, F')$, where Q is a finite set of states, G is the alphabet, T is the set of transitions of the form $q \xrightarrow{\gamma} S$, where $q \in Q$, $S \subseteq Q$, and $\gamma \in G$, $I \subseteq Q$ is the set of initial states, and $F' \subseteq Q$ is the set of accepting states.*

We define the reflexive-transitive closure of the $\to$ relation as follows: $q \xrightarrow{\varepsilon}{}^* q$, and $q \xrightarrow{\gamma\omega}{}^* S$ iff $q \xrightarrow{\gamma} S'$ for some $S' \subseteq Q$, $\gamma \in G$, $\omega \in G^*$, and for each $q' \in S'$, $q' \xrightarrow{\omega}{}^* S_{q'}$, and $S = \bigcup_{q' \in S'} S_{q'}$.

To compute $Y_{\mathcal{BP}}$, we construct an AMA $\mathcal{A} = (Q, G, T, I, F')$, such that $I \subseteq P' \times 2^\delta \times \mathbb{N}$ is the set of enumerated states of the form $(p, \theta)^i$, where $p \in P'$ is a control location in $\mathcal{BP}$, $\theta \subseteq \delta$ is a phase in $\mathcal{BP}$, $Q = I \cup \{q_f\}$, and $F' = \{q_f\}$, where q_f is a single accepting state.

We show that we can construct $\mathcal{A}$ using Algorithm 1. To force the termination of the algorithm, we define the acceleration function pr for some set of states $S \in Q$ of $\mathcal{A}$, defined as:

$$pr^{-1}(S) = \begin{cases} \{q^k \mid q^{k+1} \in S\} \cup \{q_f\} \text{ if } q_f \in S \text{ or } \exists q^1 \in S \\ \{q^k \mid q^{k+1} \in S\} \text{ else} \end{cases}$$

$$pr^k(S) = \{q^k \mid \exists j, 1 \leq j \leq k : q^j \in S\} \cup \{q_f \mid q_f \in S\}$$

The $pr^{-1}(S)$ function projects states in S of the form q^k onto states of the form q^{k-1}, such that rules of the form q^1 are projected onto q_f. The $pr^k(S)$ function projects states of the form q^j in S for any j onto states of the form q^k without changing q_f. We can show that (the proof can be found in Appendix C):

Theorem 3. *Algorithm 1 terminates and computes an AMA $\mathcal{A}$, s.t. there is a path in $\mathcal{A}$ of the form $(p, \theta)^k \xrightarrow{w}{}^* \{q_f\}$ iff $(\langle p, w\rangle, \theta) \in Y_{\mathcal{BP}}$.*

Construction of $\mathcal{A}$ follows the computation steps of $Y_{\mathcal{BP}} = \bigcap_{i\geq 0} Y_i$. Intuitively, $\mathcal{A}$ is a kind of a collection of AMAs A_i (that contain states of the form q^i) for $i \geq 0$, such that a configuration $(\langle p, \omega\rangle, \theta) \in Y_i$ iff $(p, \theta)^i \xrightarrow{\omega}{}^* \{q_f\}$ in $\mathcal{A}$. Let the set of configurations $(\langle p, \omega\rangle, \theta)$, such that $(p, \theta)^i \xrightarrow{\omega}{}^* \{q_f\}$

Algorithm 1 Computation of $Y_{\mathcal{BP}}$

Input: $\mathcal{BP} = (P', \Gamma', X, \Delta', restore, discard, F, \delta)$.
Output: An AMA $\mathcal{A} = (Q, G, T, I, \{q_f\})$.

1: $k \coloneqq 0$, $T \coloneqq \{q_f \xrightarrow{\gamma} \{q_f\} \mid \forall \gamma \in \Gamma'\}$, $\forall p \in P'$ and $\forall \theta \subseteq \delta$, $(p,\theta)^0 \coloneqq q_f$;
2: **repeat**(we call this $loop_1$)
3: $k \coloneqq k + 1$;
4: For all $p \in F, \theta \subseteq \delta$, add $(p,\theta)^k \xrightarrow{\varepsilon} \{(p,\theta)^{k-1}\} \in T$;
5: **repeat**(we call this $loop_2$)
6: **for all** $\theta \subseteq \delta$, $p\gamma \xrightarrow{[\sigma_1,\ldots,\sigma_h]} \{p_1w_1, \ldots, p_hw_h\}$ **do**
7: let $\sigma_j = (\rho_j, \rho'_j)$ and $\theta_j = (\theta \backslash \rho_j) \cup \rho'_j$;
8: $T \coloneqq T \cup \{(p,\theta)^k \xrightarrow{\gamma} \bigcup_{R \in W} R \mid W \in \prod_{j=1}^{h} W_j\}$, s.t. $W_j = \{R_j | \rho_j \subseteq \theta, (p_j,\theta_j)^k \xrightarrow{em(\theta,w_j)}{}^{*} R_j\}$;
9: **end for**
10: **for all** $p\chi \hookrightarrow p'\gamma \in restore$, s.t. $\exists (p',\theta) \xrightarrow{\gamma} R$ **do**
11: $T \coloneqq T \cup \{(p,\theta')^k \xrightarrow{\chi^\theta} R \mid \theta' \subseteq \delta\}$;
12: **end for**
13: **for all** $p\chi \hookrightarrow p'\gamma \in discard$, s.t. $\exists (p',\theta) \xrightarrow{\gamma} R$ **do**
14: $T \coloneqq T \cup \{(p,\theta)^k \xrightarrow{\chi^{\theta'}} R \mid \theta' \subseteq \delta\}$;
15: **end for**
16: **until** No new transitions can be added;
17: For all $p \in F$, $\theta \subseteq \delta$, remove $(p,\theta)^k \xrightarrow{\varepsilon} \{(p,\theta)^{k-1}\}$ from T;
18: For all $p \in P'$, $\theta \subseteq \delta$, $\gamma \in G$, $R \subseteq Q$, replace $(p,\theta)^k \xrightarrow{\gamma} R$ by $(p,\theta)^k \xrightarrow{\gamma} pr^k(R)$;
19: **until** $k > 1$ and $\forall p \in P', \gamma \in G, \theta \subseteq \delta, R \subseteq P' \times 2^\delta \times \{k\} \cup \{q_f\}, (p,\theta)^k \xrightarrow{\gamma} R \in T \iff (p,\theta)^{k-1} \xrightarrow{\gamma} pr^{-1}(R) \in T$;

in $\mathcal{A}$ at a given point of execution be denoted as $L(A_i)$. The algorithm consists of two loops: $loop_1$ and $loop_2$. After the i-th iteration of $loop_1$, $\mathcal{A}$ contains states of the form q^i, such that there is a path $(p,\theta)^i \xrightarrow{w}{}^{*} \{q_f\}$ iff $(\langle p, w\rangle, \theta) \in Y_i$. This is ensured as follows. First, since $Y_0 = P' \times G^* \times \delta$, Line 1 creates transitions such that $Y_0 = L(A_0)$. Then, for each iteration i of $loop_1$, Line 4 adds such transitions that $L(A_i) = L(A_{i-1}) \cap F \times G^* \times 2^\delta$, which corresponds to $Y_{i-1} \cap F \times G^* \times 2^\delta$. Then, $loop_2$ computes the set of predecessors pre^*. Intuitively, the loop on line 6 iterates over rules of the form $p\gamma \xrightarrow{[\sigma_1,\ldots,\sigma_h]} \{p_1w_1, \ldots, p_hw_h\}$, and a phase θ, and if for each $1 \leq j \leq h$, $\sigma_j = (\rho_j, \rho'_j)$, either $\rho_j \not\subseteq \theta$, or A_i accepts the configuration $(\langle p_j, w_j\omega\rangle, \theta_j)$, where $\theta_j = (\theta \backslash \rho_j) \cup \rho'_j$ and $\omega \in G^*$, then it means that each successor obtained by applying this rule is accepted in A_i. Therefore, the loop creates a transition, such that $(\langle p, \gamma\omega\rangle, \theta)$ is also accepted in A_i. The loop on line 10 ensures that if there is a rule of the form $p\chi \xrightarrow{restore} p'\gamma$, and a configuration of the form $(\langle p', \gamma\omega\rangle, \theta)$, for some ω, is accepted in A_i, then, for any phase θ', the predecessor $(\langle p, \chi^\theta\omega\rangle, \theta')$ must also be accepted in A_i. The loop on line 13 ensures that if there is a rule of the form $p\chi \xrightarrow{discard} p'\gamma$, and a configuration of the form $(\langle p', \gamma\omega\rangle, \theta)$, for some ω, is accepted in A_i, then, for any phase θ', the predecessor $(\langle p, \chi^{\theta'}\omega\rangle, \theta)$ must also be accepted in A_i.

Thus, $loop_2$ computes the set of predecessors pre^*, such that after it finishes, $L(A_i) = pre^*(L(A_{i-1}) \cap F \times G^* \times 2^\delta)$, which corresponds to $pre^*(Y_{i-1} \cap F \times G^* \times 2^\delta)$. Then, Line 17 removes the initial rules added on Line 4. Since these rules were added before $loop_2$, we obtain that $L(A_i) = pre^+(L(A_{i-1}) \cap F \times G^* \times 2^\delta)$, which corresponds to $Y_i = pre^+(Y_{i-1} \cap F \times G^* \times 2^\delta)$.

Therefore, from Theorems 1, 2, and 3, we can show that:

Theorem 4. *We can construct an AMA $\mathcal{A}$, such that for any configuration $(\langle p, w\rangle, \theta)$ of $\mathcal{P}$ and any BranchCARET formula ψ, $(\langle p, w\rangle, \theta) \models \psi$ iff $([p, \psi], \theta) \xrightarrow{w}{}^{*} \{q_f\}$ in $\mathcal{A}$.*

E. Model Checking with Regular Valuations

Formulas specifying malicious behaviour presented in Section III-A use propositions that talk about stack contents. For example, we use proposition $d\Gamma^*$ to state that d is on top of the stack. We show in this section how we can extend our BranchCARET model checking approach to facilitate such atomic propositions. We can describe such propositions using regular languages:

Definition 5. *For an SM-PDS $\mathcal{P} = (P, \Gamma, \Delta)$, a set of configurations W is regular if: $(\langle p, \omega\rangle, \theta) \in W$ iff $\omega \in L_p$, where $L_p \subseteq \Gamma^*$ is a regular set for each $p \in P$.*

A *regular* valuation is a function $\nu : AP \to 2^{Conf}$ that assigns to each atomic proposition a regular set of configurations. To incorporate regular valuations into the model checking process, we can extend the construction algorithm given in Section IV-B. Let $\mathcal{M}_p$ be a finite automaton that accepts L_p. The extension is straightforward and follows the lines of [1], which consists in popping the whole stack according to the transitions in $\mathcal{M}_p$ when $\mathcal{BP}$ is at the control location of the form $[p, a]$, where $a \in AP$. Intuitively, suppose that the regular valuation maps the atomic proposition a to a configuration $(\langle p, \gamma_1 \ldots \gamma_k \#\rangle, \theta)$, i.e. the automaton $\mathcal{M}_p$ accepts the word $\gamma_1 \ldots \gamma_k$. That is, there is path in $\mathcal{M}_p$ of the form $q_1 \xrightarrow{\gamma_1} q_2 \xrightarrow{\gamma_2} \ldots \xrightarrow{\gamma_k} q_f$, where $q_1, q_2, \ldots q_f$ are states of $\mathcal{M}_p$, and q_f is an accepting state. This would correspond to the execution of $\mathcal{BP}$ of the form $(\langle [p, a], \gamma_1 \ldots \gamma_k \#\rangle, \theta) \Longrightarrow (\langle [q_1, a], \gamma_1 \ldots \gamma_k \#\rangle, \theta) \Longrightarrow (\langle [q_2, a], \gamma_2 \ldots \gamma_k \#\rangle, \theta) \Longrightarrow \ldots \Longrightarrow (\langle [q_k, a], \gamma_k \#\rangle, \theta) \Longrightarrow (\langle [q_f, a], \#\rangle, \theta)$. Then, we ensure that there is an infinite run from $(\langle [q_f, a], \#\rangle, \theta)$ by creating a loop of the form $[q_f, a]\# \xrightarrow{[\top]} \{[q_f, a]\#\}$. And we ensure that this run is accepting by setting $[q_f, a]$ to be an accepting control location. Therefore, there is an accepting run from $(\langle [p, a], \gamma_1 \ldots \gamma_2 \#\rangle, \theta)$ only if $(\langle p, \gamma_1 \ldots \gamma_k \#\rangle, \theta) \models a$. Thus, we can show that:

Theorem 5. *We can effectively model check SM-PDS against a BranchCARET formula with regular valuations.*

V. Experiments

We have implemented our algorithm using Zig programming language and performed practical experiments. Our experiments were conducted on a machine with Intel Core i3-4150 CPU @ 3.50GHz with 10 GB available memory. Then, we applied our approach for malware detection.

A. Our Algorithm vs. Naive Approach

It was shown that given an SM-PDS, one can construct an equivalent standard non-self-modifying PDS [2]. And an efficient algorithm for BranchCARET model checking of standard PDS was proposed in [5], [6]. Therefore, a naive approach for BranchCARET model checking of SM-PDS would be to construct an equivalent PDS, and use the existing algorithms. However, this approach is not efficient. We have implemented the naive approach and compared it with our algorithm. We generated a random SM-PDS $\mathcal{P}$ and a random BranchCARET formula ψ. Then, we ran both algorithms to model check $\mathcal{P}$ against ψ. We recorded the execution time and the maximum memory consumed during execution. We repeated this process multiple times and presented the results in Table I. Our algorithm shows significant improvement in both speed and memory consumption (up to 30x faster and 40x less memory required). We highlighted in the table the cases showing significant improvements in memory or time. Moreover, when we consider large SM-PDS models ($\approx$ 800 rules) or long BranchCARET formulas ($\approx$ 7-9 operators), the naive approach fails either because of timeout, or because of lack of memory, while our algorithm finishes in seconds.

TABLE I

Comparison of performance of our approach vs. PDS approach ($|\Delta^s| + |\Delta^c|$ are numbers of standard and self-modifying rules respectively, $|\psi|$ is the size of BranchCARET formula, OOM - out of memory).

		Our approach		PDS approach	
$\lvert\Delta^s\rvert + \lvert\Delta^c\rvert$	$\lvert\psi\rvert$	**Time**	**Memory**	**T**, s	**Memory**
74 + 7	9	1.7s	37.3 MB	21.9s	591.4 MB
117 + 7	3	0.3s	10.4 MB	6.0s	362.8 MB
132 + 8	5	1.0s	23.3 MB	17.8s	540.1 MB
113 + 11	**3**	**0.6s**	**14.7 MB**	**19.4s**	**721.6 MB**
75 + 12	3	0.3s	16.0 MB	6.1s	400.2 MB
104 + 12	**9**	**4.4s**	**73.3 MB**	**46.7s**	**1.5 GB**
114 + 12	5	0.4s	18.4 MB	17.1s	838.0 MB
81 + 14	5	0.6s	23.9 MB	12.4s	687.3 MB
75 + 15	7	2.1s	40.7 MB	22.2s	830.1 MB
93 + 15	5	0.4s	17.2 MB	7.1s	651.0 MB
90 + 17	9	2.4s	53.8 MB	17.5s	766.3 MB
78 + 19	5	0.7s	14.9 MB	9.7s	601.0 MB
78 + 20	5	2.3s	70.5 MB	16.6s	920.1 MB
99 + 20	**5**	**1.1s**	**37.3 MB**	**24.4s**	**1.6 GB**
104 + 11	9	0.8s	18.9 MB	**Timeout**	**Timeout**
108 + 13	7	9.3s	91.6 MB	**OOM**	**OOM**
105 + 18	7	0.7s	27.3 MB	**OOM**	**OOM**
68 + 20	9	9.8s	35.2 MB	**Timeout**	**Timeout**
89 + 19	7	1.9s	42.7 MB	**Timeout**	**Timeout**
118 + 20	9	11.5s	116.2 MB	**OOM**	**OOM**
86 + 19	9	2.7s	52.2 MB	**Timeout**	**Timeout**
1083 + 18	5	88.4s	4.2 GB	**OOM**	**OOM**
1224 + 16	9	112.5s	3.1 GB	**OOM**	**OOM**
677 + 19	3	13.7s	661.4 MB	**OOM**	**OOM**
714 + 12	3	11.5s	748.5 MB	43.0s	3.2 GB
1390 + 8	7	43.4s	1.6 GB	**OOM**	**OOM**
851 + 14	9	135.3s	4.2 GB	**OOM**	**OOM**
1280 + 17	5	64.4s	3.9 GB	**OOM**	**OOM**

B. Application for Malware Detection

We considered self-modifying versions of malware samples taken from public databases such as Malware Bazaar [26] and Virus Share [27]. Then, we have built a tool based on our algorithm that performs malware detection. Our tool translates binary files into SM-PDS models using the method given in Section II-B. For the oracle, we used ANGR [24]. Then, the tool checks if the model satisfies one of the BranchCARET formulas specifying malicious behaviour (given in Section III-A). If it does, i.e. the model contains the malicious behaviour, the tool marked the binary as malicious. Table II shows our experiments. Our tool is able to detect several malwares, such as Trojan, Backdoors, and Spyware. In most cases, the tool was able to finish the analysis within several minutes, even for large binaries consisting of thousands of rules.

VI. Conclusion

In this work, we considered static malware detection in self-modifying binary code using BranchCARET model checking on SM-PDS. We model binary code as SM-PDS and we specify malicious behaviours using BranchCARET, and we reduce the malware detection problem to the problem of model checking SM-PDS against BranchCARET. We have proposed an efficient algorithm for BranchCARET model checking of SM-PDS. We have implemented the algorithm in a tool for model checking binary programs against BranchCARET formulas. Our approach is promising as shown by our experiments. In particular, our tool is able to detect several malicious behaviours.

References

[1] F. Song and T. Touili, "Efficient CTL model-checking for pushdown systems," *Theoretical Computer Science*, 2014.

[2] T. Touili and X. Ye, "LTL model checking of self modifying code," *Formal Methods in System Design*, 2022.

[3] R. Alur, K. Etessami, and P. Madhusudan, "A temporal logic of nested calls and returns," in *TACAS*, 2004.

[4] H.-V. Nguyen and T. Touili, "CARET model checking for malware detection," in *SPIN*, 2017.

[5] ——, "Branching temporal logic of calls and returns for pushdown systems," in *IFM*, 2018.

[6] J. O. Gutsfeld, M. Müller-Olm, and B. Nordhoff, "A branching time variant of CaRet," in *SPIN*, 2018.

[7] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, 1976.

[8] R. David, S. Bardin, T. D. Ta, L. Mounier, J. Feist, M.-L. Potet, and J.-Y. Marion, "Binsec/se: A dynamic symbolic execution toolkit for binary-level analysis," in *SANER*, 2016.

[9] V. Chipounov, V. Kuznetsov, and G. Candea, "S2e: A platform for in-vivo multi-path analysis of software systems," *Acm Sigplan Notices*, 2011.

[10] K. Gorna, N. Iooss, Y. Seurin, and R. Khatoun, "Zorya: Automated concolic execution of single-threaded go binaries," *SAC*, 2026.

[11] J. Kinder, S. Katzenbeisser, C. Schallhart, and H. Veith, "Detecting malicious code by model checking," in *DIMVA*, 2005.

[12] P. Beaucamps, I. Gnaedig, and J.-Y. Marion, "Abstraction-based malware analysis using rewriting and model checking," in *ESORICS*, 2012.

TABLE II
MALWARE DETECTION RESULTS.

Malware	Rules #	ψ	Time
Virus.Win32.Spit.c.exe	1332	ψ_s	581.12s
Trojan-Downloader.Win32.Small.axw.exe	1376	ψ_s	89.28s
P2P-Worm.Win32.Compux.a.exe	355	ψ_s	19.46s
IRC-Worm.Win32.Azaco.c.exe	425	ψ_s	23.06s
Trojan-Downloader.Win32.Small.bdw.exe	1568	ψ_s	124.15s
Virus.Win32.Mooder.l.exe	623	ψ_s	59.90s
Virus.Win32.Smog.e.exe	127	ψ_s	2.76s
Backdoor.Win32.SubSeven.22.b2.exe	2238	ψ_s	590.01s
Trojan-Downloader.Win32.Small.hj.exe	1729	ψ_t	387.94s
Trojan-Downloader.Win32.Small.xg.exe	743	ψ_t	806.43s
Trojan-Downloader.Win32.Small.cho.exe	659	ψ_t	241.60s
Trojan.Win32.KillAV.bx.exe	457	ψ_k	18.85s
Virus.Win32.Mooder.i.exe	623	ψ_s	32.55s
Trojan-Spy.Win32.Cusrev.a.exe	557	ψ_k	15.31s
Virus.Win32.Yopper.exe	92	ψ_s	1.15s
Virus.Win32.Invictus.exe	140	ψ_s	1.05s
Virus.Win32.Morgoth.2560.exe	106	ψ_s	0.85s
Virus.Win32.Mooder.a.exe	623	ψ_s	32.94s
Virus.Win32.Kanban.a.exe	140	ψ_s	1.06s
Trojan-Downloader.Win32.Botten.exe	726	ψ_t	2785.14s
Trojan.Win32.StartPage.ay.exe	3489	ψ_k	252.44s
Trojan-Downloader.Win32.Sherlol.exe	1294	ψ_t	1360.80s
Virus.Win32.Mooder.g.exe	236	ψ_s	4.08s
Trojan.Win32.LOADER.WPW.exe	298	ψ_t	13.80s
Trojan-Downloader.Win32.Small.bdc.exe	539	ψ_t	39.18s
Trojan-Downloader.Win32.Small.ef.exe	1615	ψ_t	269.74s
Trojan-Downloader.Win32.Harnig.e.exe	1921	ψ_t	866.80s
Trojan.Win32.KillAV.bb.exe	657	ψ_k	163.42s
Trojan-Dropper.Win32.Small.ir.exe	650	ψ_k	33.48s
Virus.Win32.Mooder.d.exe	236	ψ_s	6.70s
Virus.Win32.Mooder.c.exe	623	ψ_s	53.07s
Backdoor.Win32.DTR.a.exe	455	ψ_t	135.22s
Trojan-Downloader.Win32.Harnig.a.exe	2396	ψ_t	1589.98s
Trojan.Win32.Avkillah.a.exe	4986	ψ_k	441.23s
Virus.Win32.Mooder.b.exe	623	ψ_s	52.52s
Trojan-Dropper.Win32.Small.kj.exe	1090	ψ_k	3471.64s
Trojan-Downloader.Win32.Small.fn.exe	1760	ψ_t	386.02s
Trojan.Win32.KillAV.bc.exe	457	ψ_k	80.71s
Virus.Win32.Fela.8192.exe	97	ψ_s	0.72s
Trojan-Downloader.Win32.Aphex.10.d.exe	662	ψ_k	167.18s
Trojan-Downloader.Win32.Harnig.av.exe	1197	ψ_t	940.48s
Trojan.Win32.OpenPort.c.exe	373	ψ_t	101.40s

[13] B. Blackham and G. Heiser, "Sequoll: a framework for model checking binaries," in *RTAS*, 2013.

[14] P. Battista, F. Mercaldo, V. Nardone, A. Santone, C. A. Visaggio *et al.*, "Identification of android malware families with model checking." in *ICISSP*, 2016.

[15] H. Cai, Z. Shao, and A. Vaynberg, "Certified self-modifying code," in *PLDI*, 2007.

[16] S. K. Coogan, K. P. Debray, and G. M. Townsend, "On the semantics of self-unpacking malware code," *Citeseer, Tech. Rep.*, 2008.

[17] G. Bonfante, J.-Y. Marion, and D. Reynaud-Plantey, "A computability perspective on self-modifying programs," in *SEFM*, 2009.

[18] S. Blazy, V. Laporte, and D. Pichardie, "Verified abstract interpretation techniques for disassembling low-level self-modifying code," *Journal of Automated Reasoning*, 2016.

[19] B. Anckaert, M. Madou, and K. De Bosschere, "A model for self-modifying code," in *IH*, 2007.

[20] G. Bonfante, J. Fernandez, J.-Y. Marion, B. Rouxel, F. Sabatier, and A. Thierry, "Codisasm: Medium scale concatic disassembly of self-modifying binaries with overlapping instructions," in *CCS*, 2015.

[21] D. Korczynski, "Repeconstruct: reconstructing binaries with self-modifying code and import address table destruction," in *MALWARE*, 2016.

[22] Hex-Rays, "Ida pro," https://hex-rays.com/ida-pro, accessed: 2025-01-15.

[23] S. Àlvarez, "The Official Radare2 Book," https://book.rada.re/credits/credits.html, accessed: 2025-01-15.

[24] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel *et al.*, "SoK:(state of) the art of war: Offensive techniques in binary analysis," in *SP*, 2016.

[25] F. Song and T. Touili, "Efficient malware detection using model-checking," in *International Symposium on Formal Methods*, 2012.

[26] abuse.ch, "Malware Bazaar," https://bazaar.abuse.ch/about/, accessed: 2025-04-20.

[27] Corvus Forensics, "Virus Share," https://virusshare.com/about, accessed: 2025-04-20.

APPENDIX

A. Proof of Theorem 1

Proof. Let us define a function $Assoc$, which associates a formula $\phi \in Cl(\psi)$ and the i-th step on a run π of $\mathcal{P}$ with a configuration $(\langle [p,\phi], w'\rangle, \theta)$ of $\mathcal{BP}$. $Assoc(\phi,\pi,i) = (\langle [p,\phi], w'\rangle, \theta)$, s.t. if $w = \gamma_1 \dots \gamma_n$, then $w' = \gamma'_1 \dots \gamma'_n$, where for $1 \leq i \leq n$, $\gamma'_i = \lfloor \gamma_i, p_0, \gamma_0 \rfloor^{\theta_0}$ if γ_i was pushed by a rule of form $p_0\gamma_0 \xhookrightarrow{call} p'\gamma'_0\gamma_i$ at phase θ_0, and $\gamma'_i = \gamma_i$ otherwise.

We prove the theorem by showing that for any run π in $\mathcal{P}$, $(\pi,i) \models \phi$ iff there is an accepting run in $\mathcal{BP}$ starting from $Assoc(\phi,\pi,i)$. The proof is by induction on ϕ:

- **Base:**
  1) $\phi = a \in AP$. Let $\pi(i) = (\langle p, \gamma w\rangle, \theta)$. $(\pi,i) \models a$ iff $p \in \nu(a)$ iff $\exists [p,a]\gamma \xhookrightarrow{[\top]} \{[p,a]\gamma\} \in \Delta'$ (by item 1) iff there is an run visiting $[p,a]$ infinitely many times iff there is an accepting run starting from $Assoc(a,\pi,i)$ (the run is accepting because $[p,a] \in F$)
  2) $\phi = \neg a \in AP$. Let $\pi(i) = (\langle p, \gamma w\rangle, \theta)$. $(\pi,i) \models \neg a$ iff $p \notin \nu(a)$ iff $\exists [p,\neg a]\gamma \xhookrightarrow{[\top]} \{[p,\neg a]\gamma\} \in \Delta'$ (by item 2) iff there is an run visiting $[p,\neg a]$ infinitely many times iff there is an accepting run starting from $Assoc(\neg a,\pi,i)$ (the run is accepting because $[p,\neg a] \in F$)
- **Step:**
  1) $\phi = \phi_1 \vee \phi_2$. $(\pi,i) \models \phi$ iff $(\pi,i) \models \phi_1$ or $(\pi,i) \models \phi_2$ iff there is an accepting run from $Assoc(\phi_1,\pi,i)$ or $Assoc(\phi_2,\pi,i)$ (by induction hypothesis) iff there is an accepting run from $Assoc(\phi,\pi,i)$ (from item 3).
  2) $\phi = \phi_1 \wedge \phi_2$. Similar to case 1 using item 4.
  3) $\phi = EX^g\phi'$. Suppose $\pi(i) = (\langle p, \gamma\omega\rangle, \theta)$. $(\pi,i) \models \phi$ iff $\exists \pi' \in Ext(\pi), \pi'(i+1) = (\langle p', w\omega\rangle, \theta')$ and $(\pi', i+1) \models \phi'$, iff (1) $\exists p\gamma \xhookrightarrow{(\rho,\rho')}_t p'w \in \Delta$, s.t. $\theta' = (\theta \backslash \rho) \cup \rho'$, and (2) there is an accepting run starting from $Assoc(\phi',\pi',i+1)$ (by induction hypothesis). (1) iff $\exists [p,\phi]\gamma \xhookrightarrow{[(\rho,\rho')]} p'w' \in \Delta'$ iff $Assoc(\phi,\pi,i) \Longrightarrow \{Assoc(\phi',\pi',i+1)\}$ iff there is an accepting run from $Assoc(\phi,\pi,i)$ (from (2)).
  4) $\phi = AX^g\phi'$. The proof is similar to case 3.
  5) $\phi = EX^a\phi'$. Suppose $\pi(i) = (\langle p, \gamma\omega\rangle, \theta)$. $(\pi,i) \models \phi$ iff $\exists \pi' \in Ext(\pi), \pi'(next^a_{\pi'}(i)) = (\langle p', w\omega\rangle, \theta')$ and $(\pi', next^a_{\pi'}(i)) \models \phi'$, iff either:

– $next^a_{\pi'}(i) = i+1$ iff (1): $\exists p\gamma \xrightarrow{(\rho,\rho')}_{int} p'w \in \Delta$, s.t. $\theta' = (\theta \backslash \rho) \cup \rho'$, and (2): there is an accepting run starting from $Assoc(\phi', \pi', i+1)$ (by induction hypothesis). (1) iff $\exists [p,\phi]\gamma \xrightarrow{[(\rho,\rho')]} p'w' \in \Delta'$ iff $Assoc(\phi,\pi,i) \Longrightarrow \{Assoc(\phi',\pi',i+1)\}$ iff there is an accepting run from $Assoc(\phi,\pi,i)$ (from (2)); or

– $next^a_{\pi'}(i) = k > i+1$ iff (1): $\exists p\gamma \xrightarrow{(\rho,\rho')}_{call} p'\gamma_1\gamma_2$ and (2): there is a path $(\langle p', \gamma_1\gamma_2\omega\rangle, \theta) \Longrightarrow^* (\langle p'', \gamma_2\omega\rangle, \theta')$, s.t. $\pi'(next^a_{\pi'}(i)) = (\langle p'', \gamma_2\omega\rangle, \theta')$. (1) iff $\exists [p,\phi]\gamma \xrightarrow{[(\rho,\rho')]} \{[p',\phi]^r\gamma_1\gamma_2^r\} \in \Delta'$ (from item 7) iff $(\langle [p,\phi], \gamma\omega'\rangle, \theta) \Longrightarrow \{(\langle [p',\phi]^r, \gamma_1\gamma_2^r\omega'\rangle, \theta)\}$. Then, (2) iff (3): $(\langle [p',\phi]^r\gamma_1\gamma_2^r\omega'\rangle, \theta) \Longrightarrow \{(\langle [p'',\phi]^r\gamma_2^r\omega'\rangle, \theta')\}$. Then, from item 11, (4): $(\langle [p'',\phi]^r\gamma_2^r\omega'\rangle, \theta') \Longrightarrow \{(\langle [p'',\phi']\gamma_2\omega'\rangle, \theta')\}$. Thus, ((1) and (2)) iff ((3) and (4)) iff $Assoc(\phi,\pi,i) \Longrightarrow^* Assoc(\phi',\pi', next^a_{\pi'}(i))$ iff there is an accepting run from $Assoc(\phi,\pi,i)$.

6) $\phi = AX^a\phi'$. The proof is similar to case 5.

7) $\phi = E[\phi_1 U^g \phi_2]$. $(\pi,i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi,i)$, s.t. $\exists k \geq i$, $(\pi',k) \models \phi_2$ and for each $i \leq j < k$, $(\pi', j) \models \phi_1$. We prove by induction on $k-i$:
   – **Base** $k = i$. (1) iff $(\pi,i) \models \phi_2$ iff there is an accepting run starting from $Assoc(\phi_2,\pi,i) = (\langle [p,\phi_2], w\rangle, \theta)$ (from induction on ϕ) iff there is an accepting run from $(\langle [p,\phi], w\rangle, \theta)$ (from rule added by item 12).
   – **Step** $k > i$. (1) iff (2): $(\pi,i) \models \phi_1$ and (3): $(\pi', i+1) \models \phi$. (2) iff there is an accepting run from $Assoc(\phi_1,\pi,i) = (\langle [p,\phi_1], \gamma\omega\rangle, \theta)$ (by induction on ϕ). (3) iff there is an accepting run from $Assoc(\phi,\pi', i+1) = (\langle [p',\phi], w\omega\rangle, \theta')$ (by induction on $k-i$). $\exists \pi' \in Ext(\pi,i)$ and $\pi'(i+1) = (\langle p', w'\omega\rangle, \theta)$ iff $\exists p\gamma \xrightarrow{(\rho,\rho')}_t pw'$ iff $\exists [p,\phi]\gamma \xrightarrow{(\rho,\rho')} \{[p,\phi_1]\gamma, [p',\phi]w\}$ (from rule added by item 14) iff there is an accepting run from $Assoc(\phi,\pi,i)$ and there is no accepting runs from $Assoc(\phi_2,\pi,i)$.

8) $\phi = A[\phi_1 U^g \phi_2]$. The proof similar to case 7.

9) $\phi = E[\phi_1 U^a \phi_2]$. $(\pi,i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi,i)$, s.t. there is a sequense $s = s_0 s_1 \dots s_k$, $s_0 = i$, for any $j \geq 0$, $s_{j+1} = next^a_{\pi'}(s_j)$, $(\pi', s_k) \models \phi_2$ and for each $0 \leq j < k$, $(\pi', s_j) \models \phi_1$. We prove by induction on k:
   – **Base** $k = 0$. (1) iff $(\pi,i) \models \phi_2$ iff there is an accepting run starting from $Assoc(\phi_2,\pi,i) = (\langle [p,\phi_2], w\rangle, \theta)$ (from induction on ϕ) iff there is an accepting run from $(\langle [p,\phi], w\rangle, \theta)$ (from rule added by item 12).
   – **Step** $k > 0$. (1) iff (2): $(\pi,i) \models \phi_1$ and (3): $(\pi', next^a_{\pi'}(i)) \models \phi$. (2) iff there is an accepting run from $Assoc(\phi_1,\pi,i) = (\langle [p,\phi_1], \gamma\omega'\rangle, \theta)$ (by induction on ϕ). (3) iff there is an accepting run from $Assoc(\phi,\pi', next^a_{\pi'}(i)) = (\langle [p',\phi], w\omega'\rangle, \theta')$ (by induction on k). $\exists \pi' \in Ext(\pi,i)$ and $\pi'(next^a_{\pi'}(i)) = (\langle p', w'\omega\rangle, \theta)$ iff either:
     * $\exists p\gamma \xrightarrow{(\rho,\rho')}_{int} pw'$ iff $\exists [p,\phi]\gamma \xrightarrow{(\rho,\rho')} \{[p,\phi_1]\gamma, [p',\phi]w\}$ (from rule added by item 14) iff there is an accepting run from $Assoc(\phi,\pi,i)$, or
     * $\exists p\gamma \xrightarrow{(\rho,\rho')}_{call} p''\gamma_1\gamma_2$ and (2): there is a path $(\langle p'', \gamma_1\gamma_2\omega\rangle, \theta) \Longrightarrow^* (\langle p', \gamma_2\omega\rangle, \theta')$, s.t. $w' = \gamma_2$. (1) iff $\exists [p,\phi]\gamma \xrightarrow{[(\rho,\rho')]} \{[p'',\phi]^r\gamma_1\gamma_2^r\} \in \Delta'$ (from item 16) iff $(\langle [p,\phi], \gamma\omega'\rangle, \theta) \Longrightarrow \{(\langle [p',\phi]^r, \gamma_1\gamma_2^r\omega'\rangle, \theta)\}$. Then, (2) iff (3): $(\langle [p'',\phi]^r\gamma_1\gamma_2^r\omega'\rangle, \theta) \Longrightarrow \{(\langle [p',\phi]^r\gamma_2^r\omega'\rangle, \theta')\}$. Then, from item 18, (4): $(\langle [p',\phi]^r\gamma_2^r\omega'\rangle, \theta') \Longrightarrow \{(\langle [p',\phi]\gamma_2\omega'\rangle, \theta')\}$. Thus, ((1) and (2)) iff ((3) and (4)) iff $Assoc(\phi,\pi,i) \Longrightarrow^* Assoc(\phi,\pi', next^a_{\pi'}(i))$ iff there is an accepting run from $Assoc(\phi,\pi,i)$.

10) $\phi = A[\phi_1 U^a \phi_2]$. The proof similar to case 9.

11) $\phi = E[\phi_1 R^g \phi_2]$. $(\pi,i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi,i)$, s.t. $\forall k \geq i$, $(\pi',k) \not\models \phi_2$ implies that $\exists j < k$, $(\pi', j) \models \phi_1$. There are two cases: either $\forall k \geq i$, $(\pi',k) \models \phi_2$, or there is $k \geq i$, s.t. $(\pi,k) \models \phi_1 \wedge \phi_2$. The latter case is proven similarly to case 7.
What is left to show is that $\forall k \geq i$, $(\pi',k) \models \phi_2$ iff $\forall k \geq i$,there is an accepting run from $Assoc(\phi,\pi',k)$. We will prove in both directions:
   – ($\Longrightarrow$) We construct the accepting run from $Assoc(\phi,\pi,i) = (\langle [p,\phi], \gamma\omega'\rangle, \theta)$. We can construct a tree with the root $Assoc(\phi,\pi,i)$, and such that every branch is accepting using rules added by item 19. There is an infinite branch b with nodes of the form $(\langle [p',\phi], \omega'\rangle, \theta')$. From each of these nodes, there is an infinite branch $b_{\phi_2} = (\langle [p',\phi_2], \omega'\rangle, \theta')$. From induction on ϕ, b_{ϕ_2} is accepting. Moreover, b is also accepting because control locations of the form $[p',\phi]$ are accepting.
   – ($\Longleftarrow$) The accepting run can be constructed using rules from item 19 and represented by the tree with the root $Assoc(\phi,\pi,i)$, and each node of the form $Assoc(\phi,\pi',k)$ for $k \geq i$ having children $Assoc(\phi_2,\pi',k)$ and $Assoc(\phi,\pi',k+1)$. By induction hypothesis, $(\pi',k) \models \phi_2$ $\forall k \geq i$.

12) $\phi = A[\phi_1 R^g \phi_2]$. The proof similar to case 11.

13) $\phi = E[\phi_1 R^a \phi_2]$. $(\pi,i) \models \phi$ iff (1): $\exists \pi' \in Ext(\pi,i)$, s.t. for a sequence $s = s_0 s_1 \dots$, where $s_0 = i$, we have that $\forall k \geq 0$, $(\pi', s_k) \not\models \phi_2$ implies that $\exists j < k$, $(\pi', s_j) \models \phi_1$. There are two cases: either $\forall k \geq 0$, $(\pi', s_k) \models \phi_2$, or there is $k \geq 0$, s.t. $(\pi, s_k) \models \phi_1 \wedge \phi_2$. The latter case is proven similarly to case 9.

What is left to show is that $\forall k \geq 0$, $(\pi', s_k) \models \phi_2$ iff $\forall k \geq 0$,there is an accepting run from $Assoc(\phi, \pi', s_k)$. There are two cases: either s is finite, or s is infinite. We will prove in both directions:

- ($\Longrightarrow$) We construct the accepting run from $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma\omega' \rangle, \theta)$. We can construct a tree with the root $Assoc(\phi, \pi, i)$, and such that every branch is accepting using rules added by item 21 and item 8.

  If s is infinite, then there is an infinite branch b with infinitely many nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. From each of these nodes, there is an infinite branch $b_{\phi_2} = (\langle [p', \phi_2], \omega' \rangle, \theta')$. From induction on ϕ, b_{ϕ_2} is accepting. Moreover, b is also accepting because control locations of the form $[p', \phi]$ are accepting.

  If s is finite, it either ends with a ret statement, or an infinite loop inside a called procedure. If ret statement ended s at s_j, then there is an infinite branch b with j nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, the children of the last being nodes of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle True, \omega' \rangle, \theta')$. If an unfinished $call$ ended s at s_j, then there is an infinite branch b with j nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$, the children of the last being nodes of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle [p', \phi]^r, \omega' \rangle, \theta')$. Then, the branch starting from $(\langle [p', \phi]^r, \omega' \rangle, \theta')$ is accepting because control locations of the form $[p', \phi]^r$ are accepting and repeat infinitely many times (because the call at s_j never returns).
- ($\Longleftarrow$) The accepting run can only be constructed using rules from item 21 and 8. Let $s = s_0 s_1 \dots$ be the sequence such that $s_0 = i$ and for each $j \geq 0$, $s_{j+1} = next^a_{\pi'}(s_j)$. There are three cases:
  - there is a branch b with infinitely many nodes of the form $(\langle [p', \phi], \omega' \rangle, \theta')$. Let b_j be the j-th node of such form. First, we show that $b_j = Assoc(\phi, \pi', s_j)$. By definition, $b_0 = Assoc(\phi, \phi', s_0)$. Then, for each $j \geq 0$, if b_{j+1} has the form $(\langle [p', \phi], \omega' \rangle, \theta')$, then there is $p\gamma \xrightarrow{(\rho, \rho')}_{int} p'w \in \Delta$ and thus, $s_{j+1} = s_j + 1$. If the child of b_j has the form $(\langle [p', \phi]^r, \omega' \rangle, \theta')$, then there is $p\gamma \xrightarrow{(\rho, \rho')}_{call} p'w \in \Delta$. Similar to the case 5, we get that b reaches b_{j+1} iff $b_j = Assoc(\phi, \pi', s_{j+1})$.
  - There is the last b_j, such that the children of b_j are of the forms $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle True, \omega' \rangle, \theta')$. Then, the rule applied at s_j must be ret, and therefore, s also ends at s_j.
  - There is the last b_j, such that the children of b_j are of the forms $(\langle [p', \phi_2], \omega' \rangle, \theta')$ and $(\langle [p', \phi]^r, \omega' \rangle, \theta')$. Then, the rule applied at s_j must be $call$. Since there is no more nodes of the form $\langle [p', \phi], \omega' \rangle, \theta')$, then the corresponding return was not met, and therefore, s also ends at s_j.

    Each node b_j has another child of the form $(\langle [p', \phi_2], \omega' \rangle, \theta')$. Thus, from the induction on ϕ, we get that for each s_j, $(\pi', s_j) \models \phi_2$.

14) $\phi = A[\phi_1 R^a \phi_2]$. The proof similar to case 13.
15) $\phi = EX^c\phi'$. We consider two cases:
  - ($next^c_\pi(i)$ is defined) Suppose $next^c_\pi(i) = k$, $\pi(i) = (\langle p_i, w\omega \rangle, \theta_i)$, and $\pi(k) = (\langle p_k, \gamma_k\omega \rangle, \theta_k)$. Let the call at the k-th step would be of the form $p_k\gamma_k \xrightarrow{\sigma}_{call} p_{k+1}\gamma'\gamma''$. Hence, if $Assoc(\phi', \pi, k) = (\langle [p_k, \phi']\gamma_k\omega' \rangle, \theta_k)$, then $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi], w'\omega' \rangle, \theta_i)$, s.t. $w = u\gamma''$ and $w' = \gamma_i u \lfloor \gamma'', p_k, \gamma_k \rfloor^{\theta_k}$. Item 23 adds a rule of the form $[p_i, \phi]\gamma_i \xrightarrow{[\top]} \{[p^c, \phi]\varepsilon\}$ and item 26 adds rules of the form $[p^c, \phi]\gamma^j \xrightarrow{[\top]} \{[p^c, \phi]\varepsilon\}$ for $\gamma^j \in u$. Only these rules are allowed to be applied at each control location. Hence, $(\langle [p_i, \phi], w'\omega' \rangle, \theta_i) \Longrightarrow^* \{(\langle [p^c, \phi], \lfloor \gamma'', p_k, \gamma_k \rfloor^{\theta_k}\omega' \rangle, \theta_i)\}$. Then, only the rule from $restore$ can be applied and it allows $(\langle [p^c, \phi], \lfloor \gamma'', p_k, \gamma_k \rfloor^{\theta_k}\omega' \rangle, \theta_i) \Longrightarrow \{(\langle [p_k, \phi'], \gamma_k\omega' \rangle, \theta_k)\}$. By induction on ϕ, $(\pi, k) \models \phi'$ iff there is an accepting run from $(\langle [p_k, \phi'], \gamma_k\omega' \rangle, \theta_k) = Assoc(\phi', \pi, k)$ and hence, $(\pi, i) \models \phi$ iff there is an accepting run from $\langle [p_i, \phi], w'\omega' \rangle, \theta_i) = Assoc(\phi, \pi, i)$.
  - ($next^c_\pi(i)$ is undefined) Suppose $next^c_\pi(i) = \perp$. $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi], \gamma_i w\# \rangle, \theta_i)$, where $w \in \Gamma^*$. Item 23 adds a rule of the form $[p_i, \phi]\gamma_i \xrightarrow{[\top]} \{[p^c, \phi]\varepsilon\}$ and item 26 adds rules of the form $[p^c, \phi']\gamma^j \xrightarrow{[\top]} \{[p^c, \phi]\varepsilon\}$ for $\gamma^j \in w$. Only these rules can be applied at each control location. Thus, $(\langle [p_i, \phi], \gamma_i w\# \rangle, \theta_i) \Longrightarrow^* \{(\langle [p^c, \phi], \# \rangle, \theta_i)\}$.

    Only rule of the form $[p^c, \phi]\# \xrightarrow{[\top]} \{[p^c, \phi]\#\}$ can be applied, and since $[p^c, \phi] \notin F$, there is no accepting runs from $Assoc(\phi, \pi, i)$.
16) $\phi = E[\phi_1 U^c \phi_2]$. We prove in both directions:
  - ($\Longrightarrow$) Let $(\pi, i) \models \phi$, i.e. let $s = s_0 s_1 \dots s_k$ be the sequence, s.t. $s_0 = i$ and for $j \geq 0$, $s_{j+1} = next^c_\pi(s_j)$, and $(\pi, s_k) \models \phi_2$ and for $0 \leq j < k$, $(\pi, j) \models \phi_1$. By induction on ϕ, there are accepting runs from $Assoc(\phi_2, \pi, s_k)$ and each $Assoc(\phi_1, \pi, s_j)$. We proof by induction on k:
    - **Base** $k = 0$, then $(\pi, i) \models \phi_2$, and then, there is an accepting run from $Assoc(\phi_2, \pi, i) = Assoc(\phi_2, \pi, s_k)$. The rule added by item 12 creates allows the accepting run starting from $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_2, \pi, i)\}$.
    - **Step** $k > 0$. Then, $(\pi, i) \models \phi_1$ and $(\pi, next^c_\pi(i)) \models \phi$. By induction on ϕ,

there is an accepting from $Assoc(\phi_1, \pi, i)$, and by induction on k, there is an accepting run from $Assoc(\phi, \pi, next^c_\pi(i))$. Let $\pi(i) = (\langle p_i, \gamma_i u \gamma'' \omega \rangle, \theta_i)$, s.t. γ'' was pushed to the stack at the call locations using a rule of the form $p_{s_1}\gamma_{s_1} \overset{\sigma}{\hookrightarrow} p'\gamma'\gamma'' \in \Delta$. Thus, $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi]\gamma_i u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle^{\theta_{s_1}}, \theta_i)$, where θ_{s_1} is the phase at the call location. Rule added by item 24 creates transition $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_1, \pi, i), (\langle [p^c, \phi] u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i)\}$. Then, rules added by item 26 allow $(\langle [p^c, \phi] u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i) \Longrightarrow^* \{(\langle [p^c, \phi] \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i)\}$. Then using the $restore$ rules: $(\langle [p^c, \phi] \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i) \Longrightarrow \{(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$. And thus, $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_1, \pi, i), Assoc(\phi, \pi, next^c_\pi(i))\}$. And thus, there is an accepting run from $Assoc(\phi, \pi, i)$.

- ($\Longleftarrow$) Let $Assoc(\phi, \pi, i) = (\langle [p_i, \phi] \omega \rangle, \theta_i)$. Let k be the number of checkpoints $\chi^\theta \in G$ in ω. We prove by induction on k.
  - **Base** $k = 0$. Then, only transition $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_2, \pi, i)\}$ is possible. By induction on ϕ, $(\pi, i) \models \phi_2$ and thus, $(\pi, i) \models \phi$.
  - **Step** $k > 0$. Then, either base case applies, or the accepting run starts with $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_1, \pi, i), (\langle [p^c, \phi] u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor^{\theta_{s_1}} \omega' \rangle, \theta_i)\} \Longrightarrow^* \{Assoc(\phi_1, \pi, i), (\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$, where $\omega = \gamma u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor^{\theta_{s_1}} \omega'$. Since $(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1}) = Assoc(\phi, \pi, next^c_\pi(i))$ uses less checkpoints, by induction on k, $(\pi, next^c_\pi(i)) \models \phi$. Also, by induction on ϕ, $(\pi, i) \models \phi_1$. Thus, $(\pi, i) \models \phi$.

17) $\phi = E[\phi_1 R^c \phi_2]$. Let $s = s_0 s_1 \dots s_k$, s.t. $s_0 = i$, for $j \geq 0$, $s_{j+1} = next^c_\pi(s_j)$. We prove in both directions:

- ($\Longrightarrow$) Let $(\pi, i) \models \phi$. There are two cases: either there is j, s.t. $(\pi, s_j) \models \phi_1 \wedge \phi_2$, and for each $0 \leq l < j$, $(\pi, s_l) \models \phi_1$. In this case, the proof is similar to case 16. Or, $\forall j, 0 \leq j \leq k$, $(\pi, s_j) \models \phi_2$. In this case, we prove by induction on k:
  - **Base** $k = 0$. Then, $next^c_\pi(i) = \bot$ and if $\pi(i) = (\langle p, \gamma w \rangle, \theta)$, then w does not contain any return locations pushed by $call$ rules. Hence, $Assoc(\phi, \pi, i) = (\langle [p, \phi], \gamma w \# \rangle, \theta)$. There is a rule created by item 25, which allows $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_2, \pi, i), (\langle [p^c, \phi] w \# \rangle, \theta)\}$. And rules added by item 26 allow $(\langle [p^c, \phi] w \# \rangle, \theta) \Longrightarrow^* \{(\langle [p^c, \phi] \# \rangle, \theta)\}$. By induction on ϕ, there is an accepting run from $Assoc(\phi_2, \pi, i)$. And there is an accepting run from $(\langle [p^c, \phi] \# \rangle, \theta)$ created by the rule added by item 26 because $[p^c, \phi] \in F^c \subseteq F$.
  - **Step** $k > 0$. By induction on ϕ, there is an accepting run from $Assoc(\phi_2, \pi, i)$. Let $\pi(i) = (\langle p_i, \gamma_i u \gamma'' \omega \rangle, \theta_i)$, s.t. γ'' was pushed to the stack at the call locations using a rule of the form $p_{s_1}\gamma_{s_1} \overset{\sigma}{\hookrightarrow} p'\gamma'\gamma'' \in \Delta$. Thus, $Assoc(\phi, \pi, i)$ is of the form $(\langle [p_i, \phi] \gamma_i u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle^{\theta_{s_1}}, \theta_i)$, where θ_{s_1} is the phase at the call location. Rule added by item 25 creates transition $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_2, \pi, i), (\langle [p^c, \phi] u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i)\}$. Then, rules added by item 26 allow $(\langle [p^c, \phi] u \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i) \Longrightarrow^* \{(\langle [p^c, \phi] \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i)\}$. Then using the $restore$ rules: $(\langle [p^c, \phi] \lfloor \gamma'', p_{s_1}, \gamma_{s_1} \rfloor \omega' \rangle, \theta_i) \Longrightarrow \{(\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})\}$. By induction on k, there is an accepting run from $Assoc(\phi, \pi, next^c_\pi(i)) = (\langle [p_{s_1}, \phi] \gamma_{s_1} \omega' \rangle, \theta_{s_1})$, and since $Assoc(\phi, \pi, i) \Longrightarrow \{Assoc(\phi_2, \pi, i), Assoc(\phi, \pi, next^c_\pi(i))\}$, then there is an accepting run from $Assoc(\phi, \pi, i)$.
- ($\Longleftarrow$) There are two cases: either $Assoc(\phi, \pi, i)$ reaches some configuration $\{Assoc(\phi_1, \pi, s_j), Assoc(\phi_2, \pi, s_j)\}$. In this case, the proof is similar to case 16. Otherwise, the run reaches configurations with nodes $\{Assoc(\phi_1, \pi, s_k), (\langle [p_{s_k}, \phi] \# \rangle, \theta_{s_k})\}$. In this case, the accepting run has the form of a tree with one branch containing k nodes of the form $([p_{s_j}, \phi], \omega' \rangle, \theta_{s_j})$, and the k-th such node has children $Assoc(\phi_2, \pi, s_k)$ and $(\langle [p^c, \phi] u \# \rangle, \theta_{s_k})$. Moreover, for each s_j, there is a child of the form $Assoc(\phi_2, \pi, s_j)$ (created by the rule added by item 25). By induction on ϕ, $(\pi, s_j) \models \phi_2$ for each $0 \leq j \leq k$, and therefore, $(\pi, i) \models \phi$.

□

B. Proof of Theorem 2

Proof. First, we prove the following lemma:

Lemma 6. *$\mathcal{BP}$ has a run η starting from c, such that each path visits accepting control locations at least k times iff $c \in Y_k$.*

Proof. We prove in both directions:

- ($\Longrightarrow$) We prove by induction on k. Base case is trivial since $Y_0 = Conf_{\mathcal{BP}}$.
  Let $c \Longrightarrow^* \{(\langle p_1, w_1 \rangle, \theta_1), \dots, (\langle p_n, w_n \rangle, \theta_n)\}$, s.t. for $1 \leq i \leq n$, $(\langle p_i, w_i \rangle, \theta_i)$ is the first visited accepting node ($p \in F$). Then, for $1 \leq i \leq n$, $\mathcal{BP}$ has a run η_i starting from $(\langle p_i, w_i \rangle, \theta_i)$ and visiting accepting control locations at least $k - 1$ times. By induction

hypothesis, $(\langle p_i, w_i\rangle, \theta_i) \in Y_{k-1}$. Since $Y_{k-1} \cap (F \times G^* \times 2^\delta) \supseteq \{(\langle p_1, w_1\rangle, \theta_1), \ldots, (\langle p_n, w_n\rangle, \theta_n)\}$, then $c \in pre^*(Y_{k-1} \cap (F \times G^* \times 2^\delta)) = Y_k$.

- ($\Longleftarrow$) We prove by induction on k. Base case is trivial since $Y_0 = Conf_{\mathcal{BP}}$.
  Since $Y_k = pre^*(Y_{k-1} \cap (F \times G^* \times 2^\delta))$, then $c \Longrightarrow^* \{(\langle p_1, w_1\rangle, \theta_1), \ldots, (\langle p_n, w_n\rangle, \theta_n)\}$, where $\{(\langle p_1, w_1\rangle, \theta_1), \ldots, (\langle p_n, w_n\rangle, \theta_n)\} \subseteq Y_{k-1} \cap (F \times G^* \times 2^\delta)$, i.e. for each $1 \leq i \leq n$, $p_i \in F$ and $(\langle p_i, w_i\rangle, \theta_i) \in Y_{k-1}$. By induction hypothesis, there is a run from $(\langle p_i, w_i\rangle, \theta_i)$ visiting accepting states at least $k-1$ times. And since $p_i \in F$, then there is a run η from c that visits accepting states at least k times.

□

Now we proceed with the proof of Theorem 2. We prove in both directions:

- ($\Longrightarrow$) We show by proving that if the configuration c is not in $Y_{\mathcal{BP}}$, then there is no accepting runs from c in $\mathcal{BP}$. Suppose $c \notin Y_{\mathcal{BP}}$. Thus, there exists Y_k, s.t. $c \notin Y_k$. From Lemma 6, there is no run starting from c and visiting accepting control locations at least k times. Thus, there is no accepting runs from c.
- ($\Longleftarrow$) Suppose $c \in Y_{\mathcal{BP}}$. It is sufficient to construct an accepting run starting from c. Since the sequence $Y_0 Y_1 \ldots$ is monotone and $Y_{\mathcal{BP}} = \bigcap_{j \geq 0} Y_j$, we obtain that $Y_{\mathcal{BP}} = pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$. Since $c \in pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$, then there is a set of configurations $\{c_1, \ldots, c_n\} \subseteq Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta$, s.t. $c \Longrightarrow^* \{c_1, \ldots, c_n\}$. Therefore, for each $1 \leq i \leq n$, $c_i \in Y_{\mathcal{BP}}$.
  In other words, for $c \in Y_{\mathcal{BP}}$, we can construct a finite subrun η in $\mathcal{BP}$ with root c and leaves $\{c_1, \ldots, c_n\}$, s.t. for each $1 \leq i \leq n$, $c_i \in Y_{\mathcal{BP}}$. Then, we can extend each c_i in η with a new subrun η_i that starts at c_i and each leaf of η_i is also in $Y_{\mathcal{BP}}$. We can repeat this process infinitely many times, thus obtaining the accepting run η.

□

C. Proof of Theorem 3

Proof. The proof is mostly identical to the proof of Theorem 2 in [1]. However, we use a different ABPDS model and hence, we need to prove their Lemma 3 and Lemma 4 for SM-ABPDS.

Lemma 7 corresponds to Lemma 3 in [1]:

Lemma 7. *In Algorithm 1, for every $i \geq 1$, after line 17, we have that $\mathcal{L}(A_i) = pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.*

Proof. We prove in both directions.

($L(A_i) \subseteq pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$): Suppose $(\langle p, w\rangle, \theta) \in A_i$. There are no transitions of the form $(p, \theta) \xrightarrow{\varepsilon} \{q_f\}$ after line 17 therefore, $|w| \geq 1$.

We can show that during $loop_2$, if $(\langle p, w\rangle, \theta) \in A_i$, then $(\langle p, w\rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. We prove by induction on w.

Base $w = \varepsilon$. Then, we have a transition added at line 4, which means $p \in F$ and $(p, \theta)^i \xrightarrow{\varepsilon} \{(p, \theta)^{i-1}\}$. Therefore, respectively, $(\langle p, w\rangle, \theta) \in F \times G^* \times 2^\delta$ and $(\langle p, w\rangle, \theta) \in L(A_{i-1})$. And therefore, the base case holds.

Step There are two cases:

- $w = \gamma u$ for $\gamma \in \Gamma, u \in G^*$. Thus, there exist $S \subseteq Q$, s.t. $t = (p, \theta)^i \xrightarrow{\gamma} S \in T$, $S \xrightarrow{u}{}^* \{q_f\}$. t could be added to T at the only one place on line 8, which means $\exists p\gamma \xrightarrow{[\sigma_1, \ldots, \sigma_m]} \{p_1 w_1, \ldots, p_m w_m\}$, $\sigma_j = (\rho_j, \rho'_j)$, $S = \{q \in S_j \mid \rho_j \subseteq \theta \text{ and } (p_j, \theta_j)^l \xrightarrow{w_j}{}^* S_j\}$, where $\theta_j = (\theta \backslash \rho_j) \cup \rho'_j$. By applying the induction hypothesis, for every j, $1 \leq j \leq m$, if $\rho_j \subseteq \theta$, then $(\langle p_j, w_j u\rangle, \theta_j) \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w\rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G_i^* \times 2^\delta))$.
- $w = \chi^{\theta'} u$ for $\chi \in X, u \in G^*$, $\theta' \subseteq \delta$. Thus, there exist $S \subseteq Q$, s.t. $t = (p, \theta)^i \xrightarrow{\chi^{\theta'}} S \in T$, $S \xrightarrow{u}{}^* \{q_f\}$. t could be added to T at two places:
  - On line 11, which means $\exists p\chi \hookrightarrow p'\gamma \in restore$, $(p', \theta')^l \xrightarrow{\gamma}{}^* S$. By applying the previous item, we have that $(\langle p', \gamma u\rangle, \theta') \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w\rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G_i^* \times 2^\delta))$.
  - On line 14, which means $\exists p\chi \hookrightarrow p'\gamma \in discard$, $(p', \theta)^l \xrightarrow{\gamma}{}^* S$. By applying the previous item, we have that $(\langle p', \gamma u\rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$. Thus, we get that $(\langle p, w\rangle, \theta) \in pre^*(L(A_{i-1}) \cap (F \times G_i^* \times 2^\delta))$.

Since the algorithm removes ε transitions on line 17, we know that $|\omega| \geq 1$. We can apply the same induction but with at least one step to obtain $(\langle p, \omega\rangle, \theta) \in pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$.

($pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta)) \subseteq L(A_i)$):
Since $pre^+(W) = pre^*(pre(W))$ for any $W \subseteq Conf$, we obtain that $pre^+(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$ is the limit of the infinite sequence $\{C_j\}_{j \geq 0}$, such that $C_0 = pre(L(A_{i-1}) \cap (F \times G^* \times 2^\delta))$ and $C_{j+1} = C_j \cup pre(C_j)$ for every j, $j \geq 0$. Because $C_j \subseteq C_{j+1}$ for every $j \geq 0$, it is sufficient to show that for every tuple $(\langle p, w\rangle, \theta) \in C_j$, we have a path $(p, \theta) \xrightarrow{w}{}^* \{q_f\}$ in A_i that does not use transition rules of the form $q^i \xrightarrow{\varepsilon} \{q^{i-1}\}$ for any $q^i, q^{i-1} \in Q$. We prove by induction.

Base $j = 0$. By applying the saturation procedure at $loop_2$, A_i accepts C_0 if only the states of the form $(p, \theta)^i$ are treated as initial states, which are the outgoing states of the added transitions during $loop_2$. Since these transitions are of the form $(p, \theta)^i \xrightarrow{\gamma} S$ for $\gamma \in G$, $Q \subseteq P \times 2^\delta \times \{i-1\} \cup \{q_f\}$, we have that for every $(\langle p, w\rangle, \theta) \in C_0$, $(\langle p, w\rangle, \theta) \in L(A_i)$.

Step $j \geq 1$. Since $C_j = C_{j-1} \cup pre(C_{j-1})$, we get that either $(\langle p, w\rangle, \theta) \in C_{j-1}$ or $(\langle p, w\rangle, \theta) \in pre(C_{j-1})$. In the first case, the step holds from the induction hypothesis. Otherwise, if $(\langle p, w\rangle, \theta) \in pre(C_{j-1})$, there are two cases depending on what rule was applied:

- $\exists p\gamma \xrightarrow{[\sigma_1,\ldots,\sigma_m]} \{p_1w_m,\ldots,p_mw_m\} \in \Delta'$, such that $w=\gamma u$ and for every j, $1\le k\le m$, let $\sigma_k=(\rho_k,\rho'_k)$, if $\rho_k\subseteq\theta$, then $(\langle p_k, em(\theta,w_k)u\rangle,\theta_k)\in C_{j-1}$, where $\theta_k=(\theta\backslash\rho_k)\cup\rho'_k$. We apply the induction hypothesis to get $(p_k,\theta_k)^i \xrightarrow{em(\theta,w_k)u}{}^* \{q_f\}$. Let us deconstruct the path as follows: $(p_k,\theta_k)^i \xrightarrow{em(\theta,w_k)}{}^* Q_j \xrightarrow{u}{}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p,\theta)\xrightarrow{\gamma} S$, where $S=\{q\in Q_k \mid \rho_k\subseteq\theta\}$. Thus, we get that $(\langle p,\rangle,\theta)\in L(A_i)$.
- $\exists p\chi\hookrightarrow p'\gamma\in restore$, $w=\chi^{\theta'}u$, $(\langle p',\gamma u\rangle,\theta')\in C_{j-1}$ for some $\theta'\subseteq\delta$. We apply the induction hypothesis to get $(p',\theta')^i \xrightarrow{\gamma u}{}^* \{q_f\}$. Let us deconstruct the path as follows: $(p',\theta')^i \xrightarrow{\gamma}{}^* S \xrightarrow{u}{}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p,\theta)\xrightarrow{\chi^{\theta'}} S$. Thus, we get that $(\langle p,w\rangle,\theta)\in L(A_i)$.
- $\exists p\chi\hookrightarrow p'\gamma\in discard$, $w=\chi^{\theta'}u$, for some $\theta'\subseteq\delta$, and $(\langle p',\gamma u\rangle,\theta)\in C_{j-1}$. We apply the induction hypothesis to get $(p',\theta)^i \xrightarrow{\gamma u}{}^* \{q_f\}$. Let us deconstruct the path as follows: $(p',\theta)^i \xrightarrow{\gamma}{}^* S \xrightarrow{u}{}^* \{q_f\}$. Therefore, the saturation procedure will add the transition $(p,\theta)\xrightarrow{\chi^{\theta'}} S$. Thus, we get that $(\langle p,w\rangle,\theta)\in L(A_i)$.

Therefore, $pre^+(L(A_{i-1})\cap(F\times G^*\times 2^\delta))\subseteq L(A_i)$. □

Lemma 8 corresponds to Lemma 4 in [1]:

Lemma 8. *Consider an* **Algorithm C***, which is Algorithm 1 without line 19 (without termination condition of $loop_1$). In* **Algorithm C***, for every $i\ge 0$:*

(a) for every accepting run η in $\mathcal{BP}$ starting from $(\langle p,w\rangle,\theta)$, there is a path $(p,\theta)^i \xrightarrow{w} \{q_f\}$ in A_i and for every decomposition $(p,\theta)^i \xrightarrow{u} Q \xrightarrow{v} \{q_f\}$ of the path, if $Q\neq\{q_f\}$, then for all $(p',\theta')^i\in Q\backslash\{q_f\}$, η visits $(\langle p',v\rangle,\theta')$;

(b) After substitution at line 18, $Y_{\mathcal{BP}}\subseteq L(A_i)$.

Proof. We proceed by induction on i.

Base $i=0$. The statement (a) holds from the fact that for any $p\in P'$, $\theta\subseteq\delta$, $w\in G^*$, $(p,\theta)^0 \xrightarrow{w}{}^* \{q_f\}$ during $i=0$ and for any decomposition $(p,\theta)^i \xrightarrow{u}{}^* Q \xrightarrow{v}{}^* \{q_f\}$, $Q=\{q_f\}$. The statement (b) holds from the fact that $\mathcal{L}(A_0)=P'\times G^*\times 2^\delta$.

Step $i\ge 1$. For the statement (a). Let $H(\eta)$ be the maximum number of steps over branches of η to reach some accepting control location. We proceed by induction on $H(\eta)$.

- **Base** $H(\eta)=0$. Since the root of η is $(\langle p,w\rangle,\theta)$, then we have that $(\langle p,w\rangle,\theta)\in F\times G^*\times 2^\delta$. The line 1 adds transition $(p,\theta)^i \xrightarrow{\varepsilon} \{(p,\theta)^{i-1}\}$ and the result follows from the induction hypothesis on i.
- **Step** $H(\eta)\ge 1$. Let the root node of η, $(\langle p,w\rangle,\theta)$, have m child runs $\eta_1,\ldots,\eta_m$, s.t. for every j, $1\le j\le m$, η_j starts at $(\langle p_j,w_j\rangle,\theta_j)$. There are three cases depending on which rule was applied:
  - $\exists p\gamma \xrightarrow{[\sigma_{j_1},\ldots,\sigma_{j_h}]} \{p_{j_1}\omega_{j_1},\ldots,p_{j_h}\omega_{j_h}\} \triangleright \{D_{j_1},\ldots,D_{j_h}\}$, s.t. $\{1,\ldots,m\}\subseteq\{j_1,\ldots,j_h\}$, $w=\gamma w'$, and for every j, $1\le j\le m$, $\theta_j=(\theta\backslash\rho_j)\cup\rho'_j$, $\sigma_j=(\rho_j,\rho'_j)$, and $\rho_j\subseteq\theta$. We apply the induction on $H(\eta_j)$ for every j, $1\le j\le m$, to obtain that there is a path $(p_j,\theta_j)^i \xrightarrow{em(\theta,w_j)w'}{}^* \{q_f\}$ in A_i and for every decomposition $(p_j,\theta_j)^i \xrightarrow{u}{}^* Q_j \xrightarrow{v}{}^* \{q_f\}$, if $Q_j\neq\{q_f\}$, then for every $(p',\theta')^i$ or $(p',\theta')^{i-1}\in Q_j\backslash\{q_f\}$, η_j reaches a node labelled $(\langle p',v\rangle,\theta')$.
    Let $(p_j,\theta_j)^i \xrightarrow{w_j}{}^* Q'_j \xrightarrow{w'}{}^* \{q_f\}$. After application of the saturation at line 8, we get:
    $$(p,\theta)^i \xrightarrow{\gamma}{}^* \bigcup_{j=1}^m Q'_j \xrightarrow{w'}{}^* \{q_f\}$$
    For every decomposition $(p,\theta)^i \xrightarrow{u}{}^* Q \xrightarrow{v}{}^* \{q_f\}$, $Q\subseteq\bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p',\theta')^i$ or $(p',\theta')^{i-1}\in Q\backslash\{q_f\}$, η reaches a node labelled $(\langle p',v\rangle,\theta')$.
  - $\exists p\chi\hookrightarrow p_1\gamma\in restore$, s.t. $m=1$, $w=\chi^{\theta'}\omega$ for some θ', and $(\langle p_1,\gamma\omega\rangle,\theta')$ is the only child node. We apply the induction hypothesis on $H(\eta_1)$ to obtain that there is a path $(p_1,\theta')^i \xrightarrow{\gamma\omega}{}^* \{q_f\}$ in A_i, and for every decomposition of the form $(p_1,\theta')^i \xrightarrow{u}{}^* Q \xrightarrow{v}{}^* \{q_f\}$, if $Q\neq\{q_f\}$, then for every $(p',\theta'')^i$ or $(p',\theta'')^{i-1}\in Q\backslash\{q_f\}$, η_1 reaches a node labelled $(\langle p',v\rangle,\theta'')$.
    Let $(p_1,\theta')^i \xrightarrow{\gamma}{}^* Q' \xrightarrow{\omega}{}^* \{q_f\}$. After application of the saturation at line 8, we get:
    $$(p,\theta)^i \xrightarrow{\chi^{\theta'}}{}^* Q' \xrightarrow{\omega}{}^* \{q_f\}$$
    Therefore, since $(p,\theta)^i \xrightarrow{w}{}^* \{q_f\}$, we have that for every decomposition $(p,\theta)^i \xrightarrow{u}{}^* Q \xrightarrow{v}{}^* \{q_f\}$, $Q\subseteq\bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p',\theta'')^i$ or $(p',\theta'')^{i-1}\in Q\backslash\{q_f\}$, η reaches a node labelled $(\langle p',v\rangle,\theta'')$.
  - $\exists p\chi\hookrightarrow p_1\gamma\in discard$, s.t. $m=1$, $w=\chi^{\theta'}\omega$ for some θ', and $(\langle p_1,\gamma\omega\rangle,\theta)$ is the only child node. We apply the induction hypothesis on $H(\eta_1)$ to obtain that there is a path $(p_1,\theta)^i \xrightarrow{\gamma\omega}{}^* \{q_f\}$ in A_i, and for every decomposition of the form $(p_1,\theta)^i \xrightarrow{u}{}^* Q \xrightarrow{v}{}^* \{q_f\}$, if $Q\neq\{q_f\}$, then for every $(p',\theta'')^i$ or $(p',\theta'')^{i-1}\in Q\backslash\{q_f\}$, η_1 reaches a node labelled $(\langle p',v\rangle,\theta'')$.
    Let $(p_1,\theta)^i \xrightarrow{\gamma}{}^* Q' \xrightarrow{\omega}{}^* \{q_f\}$. After application of the saturation at line 8, we get:
    $$(p,\theta)^i \xrightarrow{\chi^{\theta'}}{}^* Q' \xrightarrow{\omega}{}^* \{q_f\}$$
    Therefore, since $(p,\theta)^i \xrightarrow{w}{}^* \{q_f\}$, we have that for every decomposition $(p,\theta)^i \xrightarrow{u}{}^* Q \xrightarrow{v}{}^* \{q_f\}$, $Q\subseteq\bigcup_{j=1}^m Q_j$, which means that for every state of the form $(p',\theta'')^i$ or $(p',\theta'')^{i-1}\in Q\backslash\{q_f\}$, η reaches a node labelled $(\langle p',v\rangle,\theta'')$.

Thus, the statement (a) holds.

For the statement (b). First, we need to show that $Y_{\mathcal{BP}}\subseteq L(A_1)$. From Lemma 7, we have that $L(A_1)=pre^+(L(A_0)\cap F\times G^*\times 2^\delta)$, i.e. $L(A_1)=pre^+(F\times G^*\times 2^\delta)$ before line 18.

Since $Y_1 = pre^+(F \times G^* \times 2^\delta)$, it is sufficient to prove that for every $k \geq 1$, $(\langle p, w\rangle, \theta) \in Y_{\mathcal{BP}}$, there is a path $(p,\theta)^k \xrightarrow{w}{}^* \{q_f\}$ in A_k after line 18. This is proven in what follows.

Suppose $k \geq 2$. We apply the induction hypothesis on k to obtain that $Y_{\mathcal{BP}} \subseteq L(A_{k-1})$.

Since $Y_{\mathcal{BP}} = pre^+(Y_{\mathcal{BP}} \cap F \times G^* \times 2^\delta)$, we get that $Y_{\mathcal{BP}} \subseteq pre^+(L(A_{k-1}) \cap F \times G^* \times 2^\delta)$. From Lemma 7, $L(A_k) = pre^+(L(A_{k-1}) \cap F \times G^* \times 2^\delta)$ after line 17. Therefore, we prove that for every $k \geq 1$, if $(\langle p, w\rangle, \theta) \in Y_{\mathcal{BP}}$, then there is a path $(p,\theta)^k \xrightarrow{w}{}^* \{q_f\}$ in A_k after line 18. Let m be the number of transitions substituted at line 18. For l, $0 \leq l \leq m$, let A_k^l be the automaton obtained after substituting l transitions. We proceed by induction on l.

- **Base** $l = 0$. We directly get that $Y_{\mathcal{BP}} \subseteq A_k^0$.
- **Step** $l \geq 1$. We apply the induction hypothesis to obtain that $Y_{\mathcal{BP}} \subseteq L(A_k^{l-1})$. If $L(A_k^{l-1}) \subseteq L(A_k^l)$, the item holds from the fact that $Y_{\mathcal{BP}} \subseteq A_k^{l-1}$. Otherwise, suppose there is $(\langle p, \omega\rangle, \theta) \in L(A_k^{l-1}) \backslash L(A_k^l)$, such that $|w| = min\{|\omega'| \mid (\langle p', \omega'\rangle, \theta') \in L(A_k^{l-1}) \backslash L(A_k^l)\}$. We prove by contradiction that $(\langle p, w\rangle, \theta) \notin Y_{\mathcal{BP}}$.
  Since $(p,\theta)^k \xrightarrow{w}{}^* \{q_f\}$ in A_k^{l-1}, let $u \in G^+$, $v \in G^*$, $p' \in P'$, $\theta' \subseteq \delta$, $Q' \subseteq Q$, such that $w = uv$, $(p,\theta) \xrightarrow{u}{}^* Q' \cup \{(p',\theta')^{k-1}\} \xrightarrow{v}{}^* \{q_f\}$ in A_k^{k-1}, and there is no such path $(p',\theta')^k \xrightarrow{v}{}^* \{q_f\}$ in A_k^l (otherwise, $(\langle p, w\rangle, \theta) \in L(A_k^l)$). By the statement (a), the accepting run must reach the local configuration $(\langle p', v\rangle, \theta')$ during the accepting run. It is sufficient to show that $(\langle p', v\rangle, \theta') \notin Y_{\mathcal{BP}}$. Since $(\langle p', v\rangle, \theta') \in L(A_k^{m-1})$, then we have that $(\langle p', v\rangle, \theta') \in L(A_k^{m-1}) \backslash L(A_k^m)$. Since $loop_2$ can only add new transitions, we get that $|v| > |w|$, which contradicts the definition of $|w|$.

Thus, the statement (b) holds.

□

□